



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

NHẬP MÔN HỌC SÂU

(Introduction to Deep Learning)

CHƯƠNG 1 : NEURAL NETWORK



GVGD: PHAN HỒ VIỆT TRƯỜNG



NỘI DUNG

01. GIỚI THIỆU HỌC SÂU

02. KHÁI NIỆM CƠ BẢN HỌC SÂU

03. MẠNG NEURAL

04. MẠNG NEURAL TÍCH CHẬP

05. MẠNG NEURAL HỒI QUY

06. MẠNG ENCODER-DECODER



GIỚI THIỆU HỌC SÂU

- Học sâu là một lĩnh vực hẹp trong Trí tuệ nhân tạo.
- Học sâu tạo các mô hình mạng nơ ron lớn có khả năng ra quyết định dựa trên dữ liệu phức tạp và lớn được cung cấp.
- Học sâu được ứng dụng hầu hết các công ty trực tuyến và công nghệ cao cấp cho người tiêu dùng.

GIỚI THIỆU HỌC SÂU

DL $\subset$ **NN** $\subset$ **ML** $\subset$ **AI**

Artificial Intelligence (AI)

- AI refers to a set of technique that enable computers to **mimic human behavior**.

Machine Learning (ML)

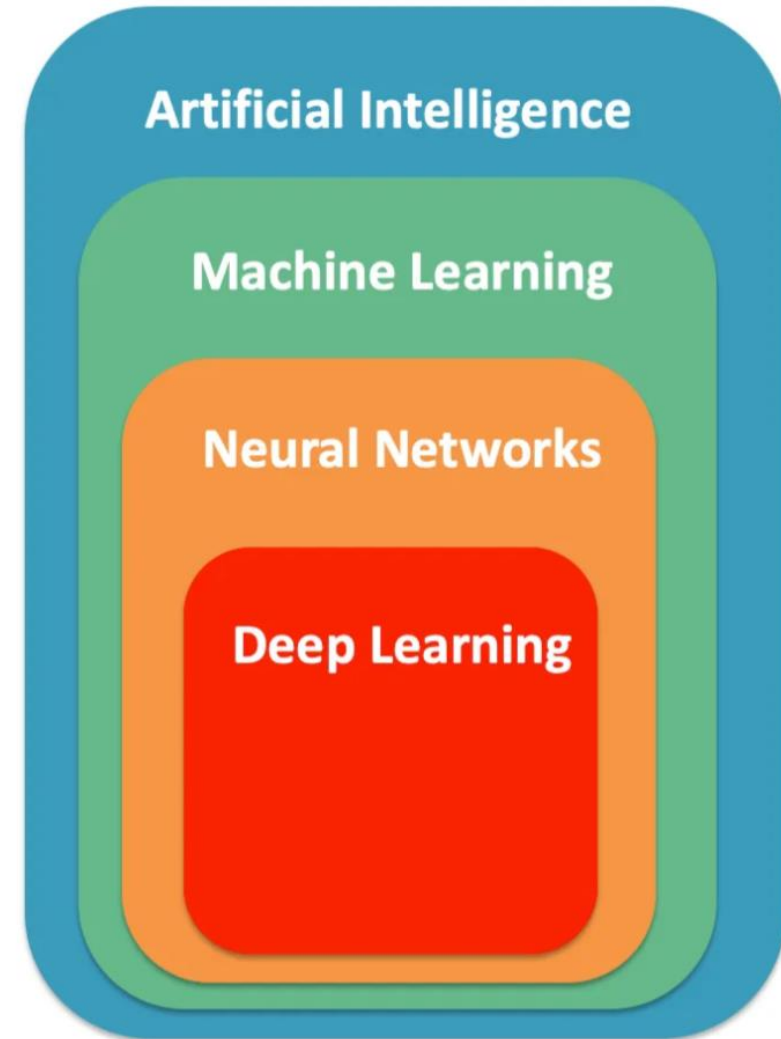
- ML is a subfield of AI , which enables machines to **learn and make prediction based on data**.

Neural Networks (NN)

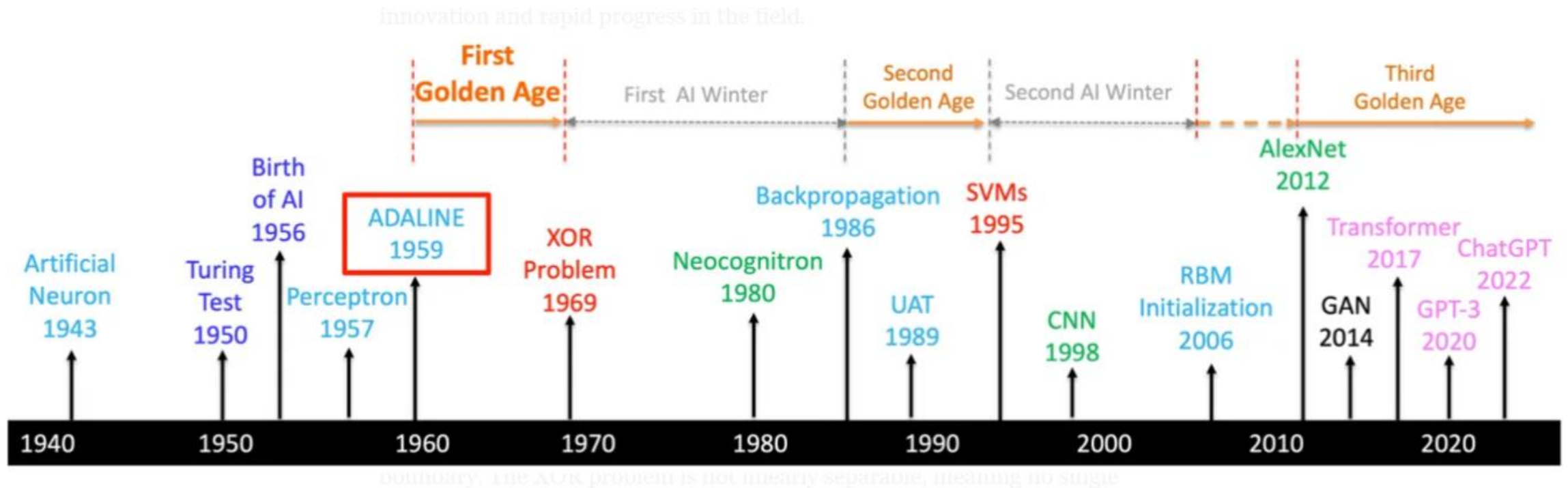
- NN are a subfield of ML that **use artificial neural networks** to extract patterns from data, inspired by the human brain.

Deep Learning (DL)

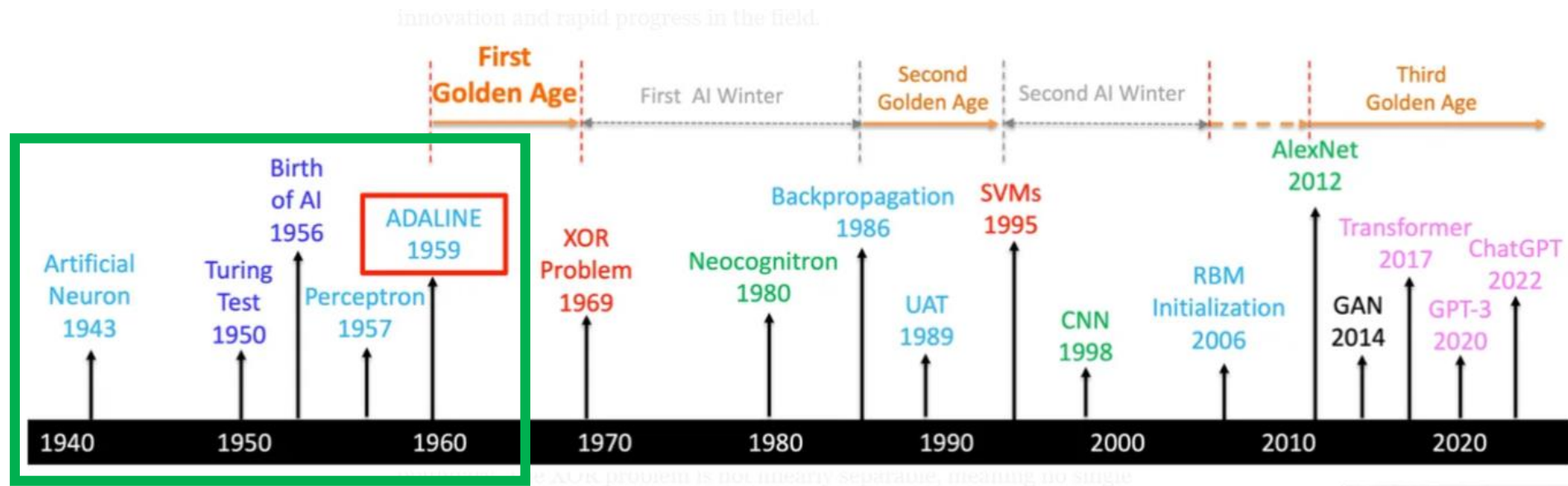
- DL is a subfield of NN that **utilizes multi-layered neural networks** to achieve high performance on complex tasks.



LỊCH SỬ PHÁT TRIỂN HỌC SÂU



LỊCH SỬ PHÁT TRIỂN HỌC SÂU



1. Muculloch-Pitts Neuron (1943)

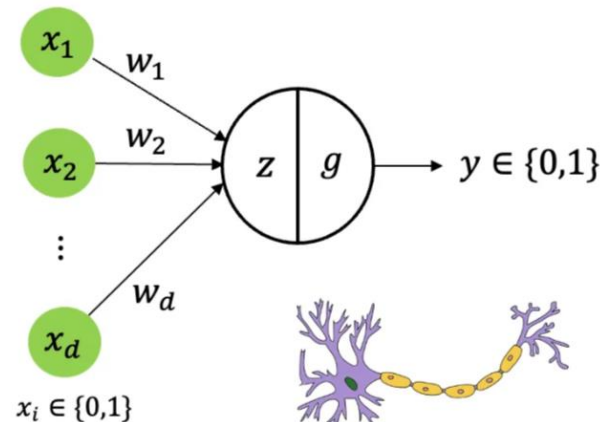
- Do Warren McCulloch và Walter Pitts đề xuất Artificial Neuron Network.

$x_i \in \{0,1\}$ and $y \in \{0,1\}$

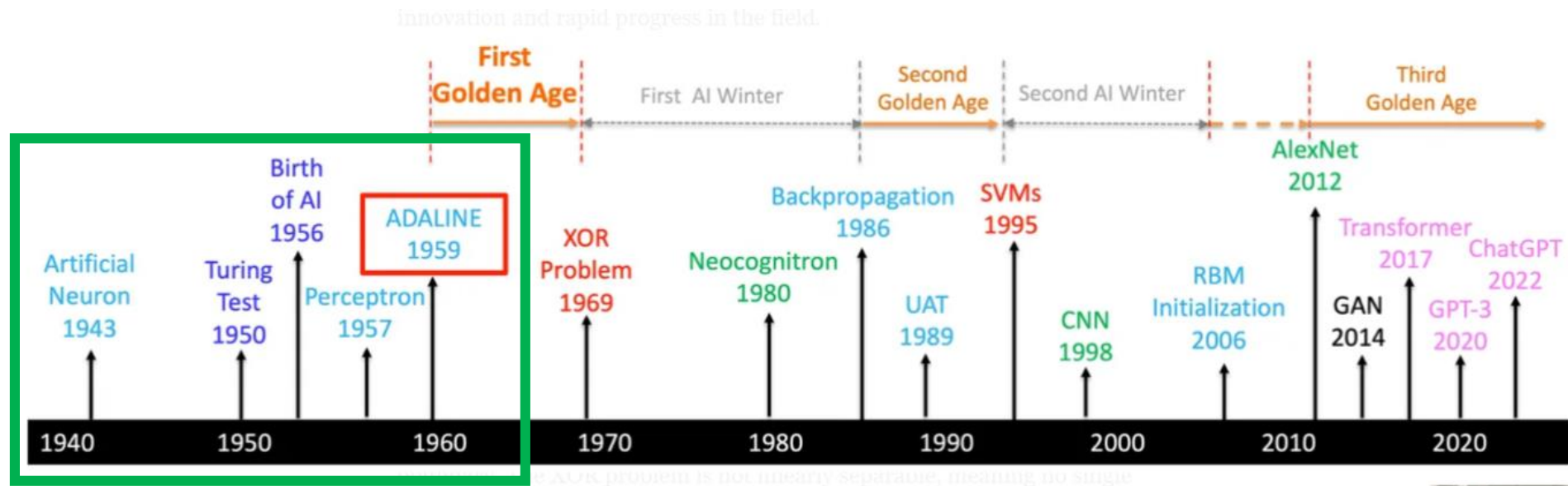
$$z(x_1, x_2, \dots, x_d) = z(\mathbf{x}) = \sum_{j=1}^d w_j \cdot x_j$$

$$y = g(z(\mathbf{x})) = \begin{cases} 1 & \text{if } z(\mathbf{x}) \geq T \\ 0 & \text{if } z(\mathbf{x}) < T \end{cases}$$

Threshold Activation
function

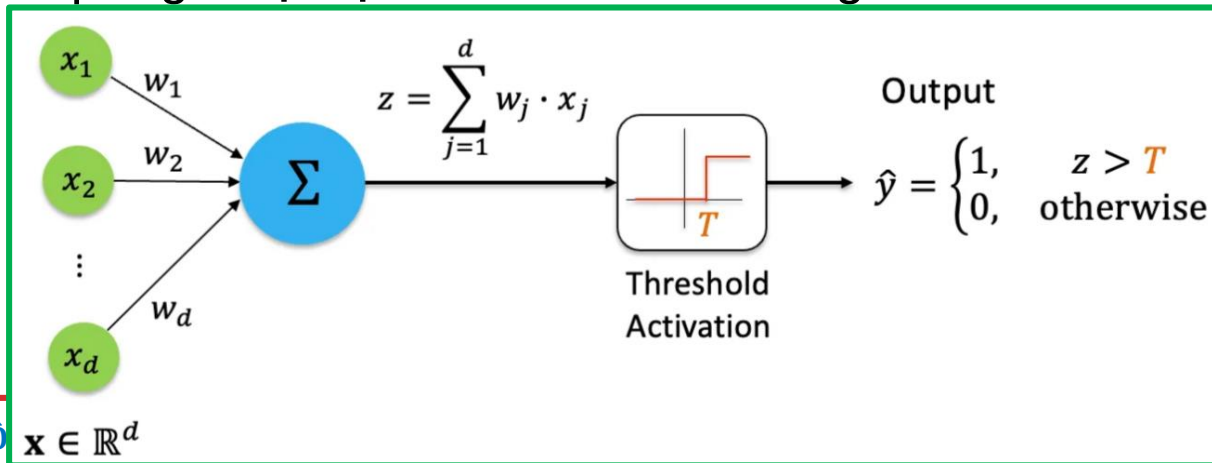


LỊCH SỬ PHÁT TRIỂN HỌC SÂU



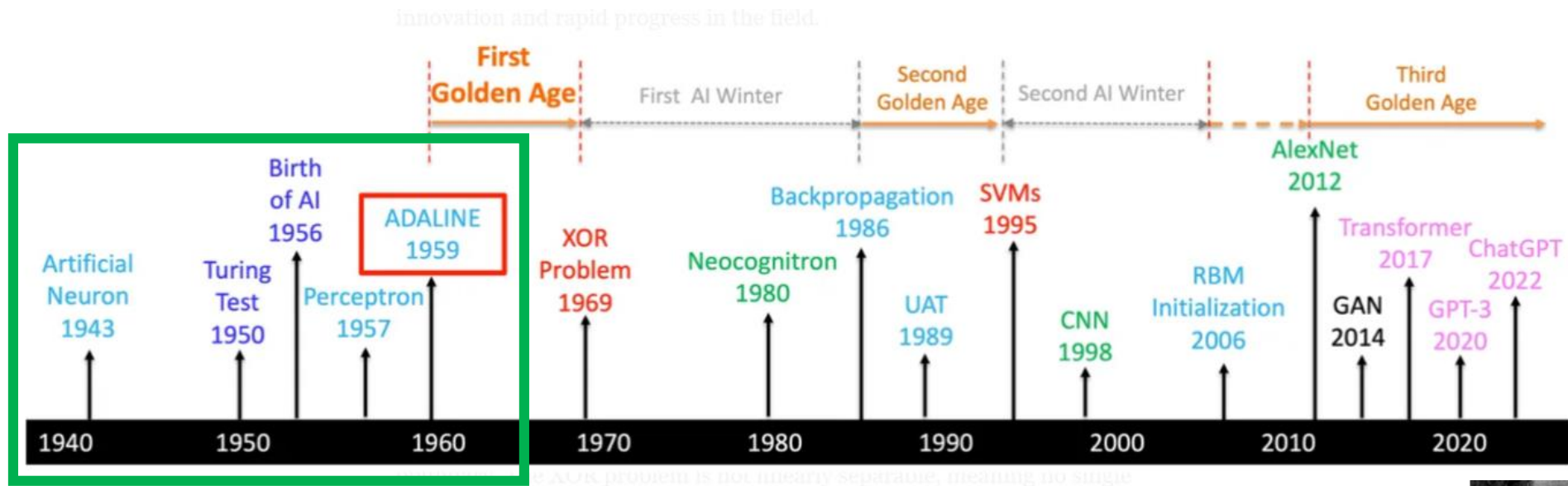
2 Rosenblatt's Perceptron Model (1957)

- Do Frank Rosenblatt đề xuất Perceptron (single NN) học và nhận dạng mẫu.
- Cho phép input giá trị thực và điều chỉnh weight



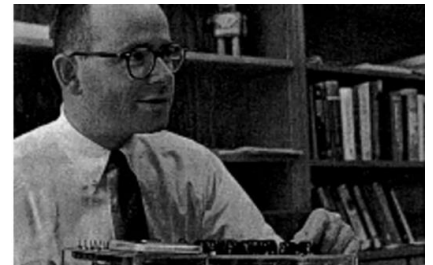
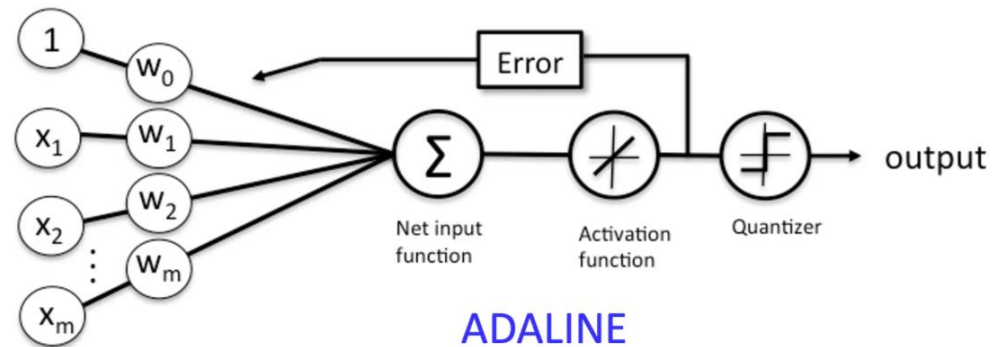
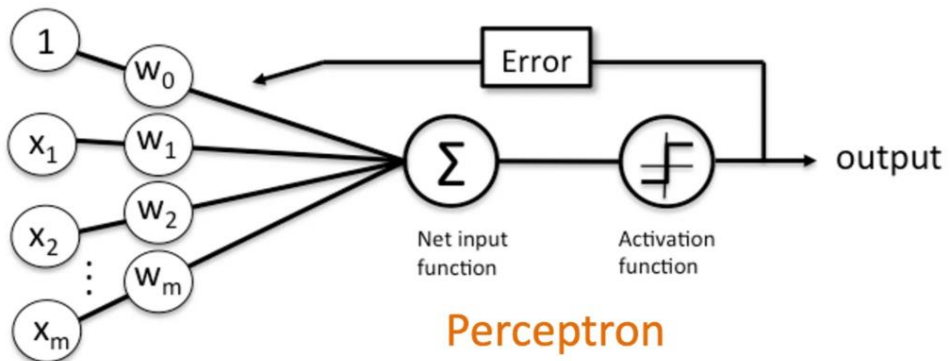
Ph Frank Rosenblatt

LỊCH SỬ PHÁT TRIỂN HỌC SÂU



3 ADALINE (1959) (1st Golden Age)

- **Widrow and Hoff** đề xuất cải tiến quá trình học Perceptron, học dữ liệu phi tuyến

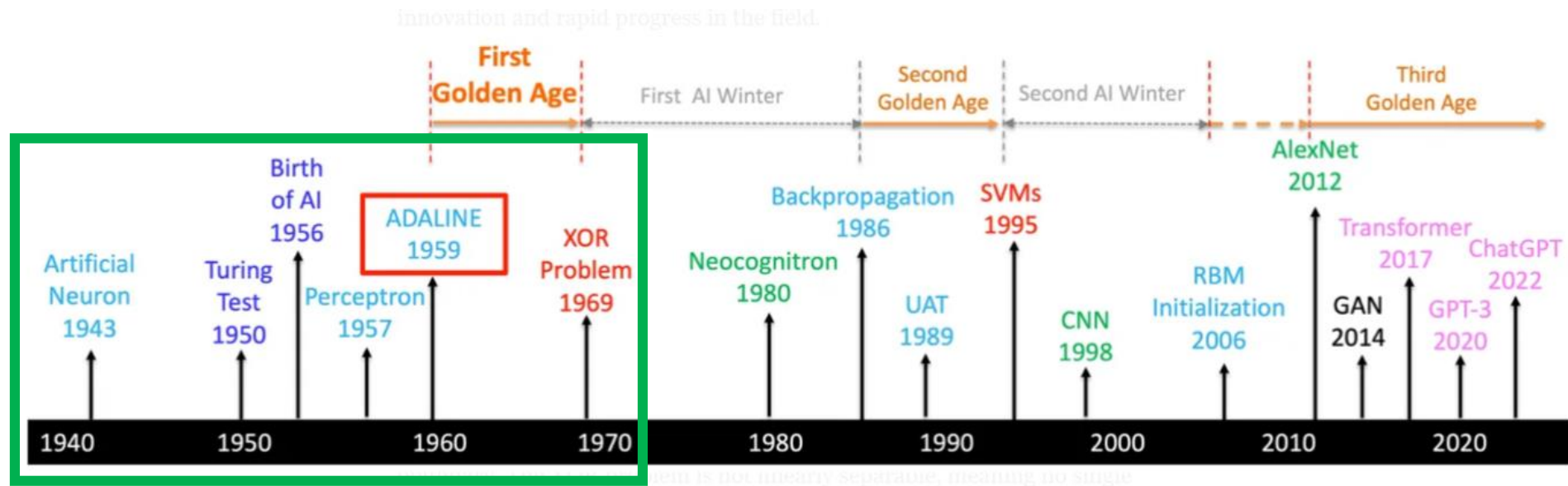


Bernard Widrow



Marcian Hoff

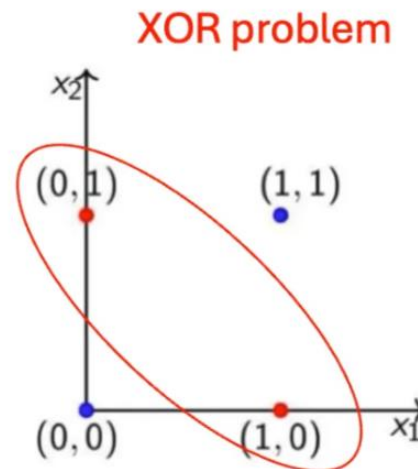
LỊCH SỬ PHÁT TRIỂN HỌC SÂU



4 The XOR Problem (1969)

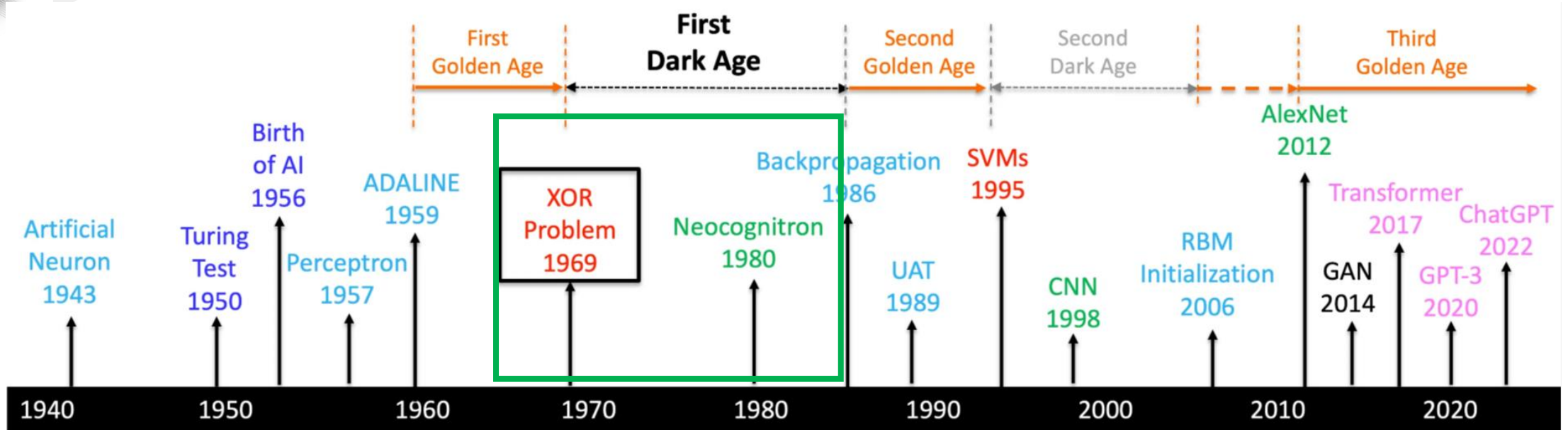
- Marvin Minsky và Seymour Papert chỉ ra hạn chế Perceptron không thể giải quyết bài toán XOR (Exclusive) trong phân lớp đơn giản. (Không có đường biên đơn tuyến tính phân lớp chính xác tất cả dữ liệu cung cấp)

Tin tức GS Papert



Minsky and Papert: Perceptrons: An introduction to computational geometry. MIT Press, 1969.

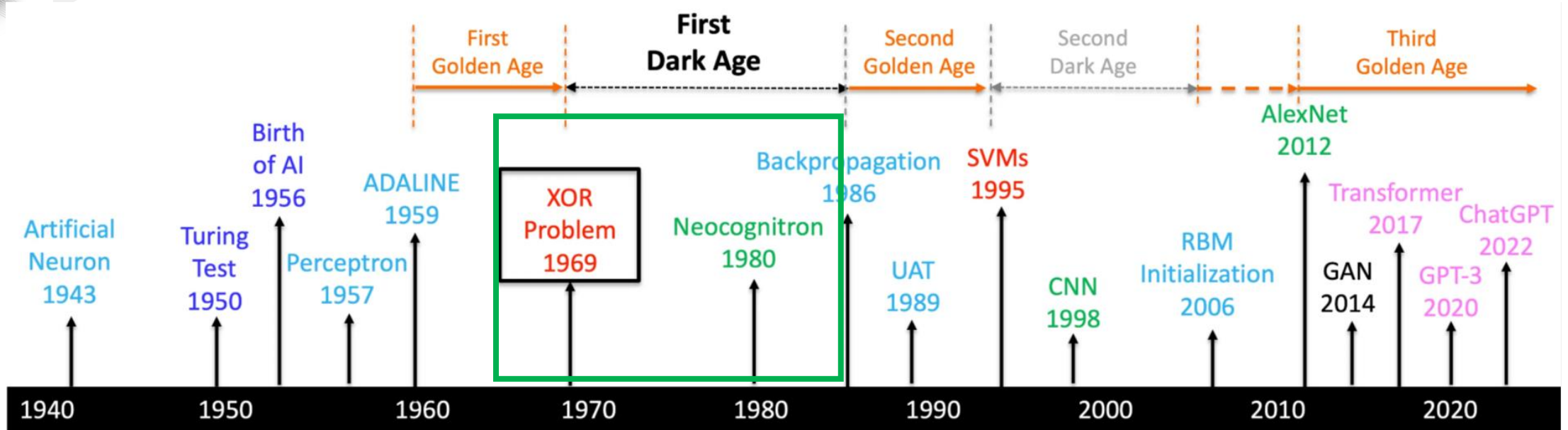
LỊCH SỬ PHÁT TRIỂN HỌC SÂU



Thời kỳ đen tối lần thứ nhất (First Dark Age)

- Vấn đề XOR làm giảm nhu cầu cần kiến trúc mạng nơ ron phức tạp cho việc học biên quyết định phi tuyến.
- Giới hạn của Perceptron làm mất niềm tin vào mạng nơ ron.
- Chuyển sang dùng các phương pháp AI symbolic (rule, frames...).

LỊCH SỬ PHÁT TRIỂN HỌC SÂU

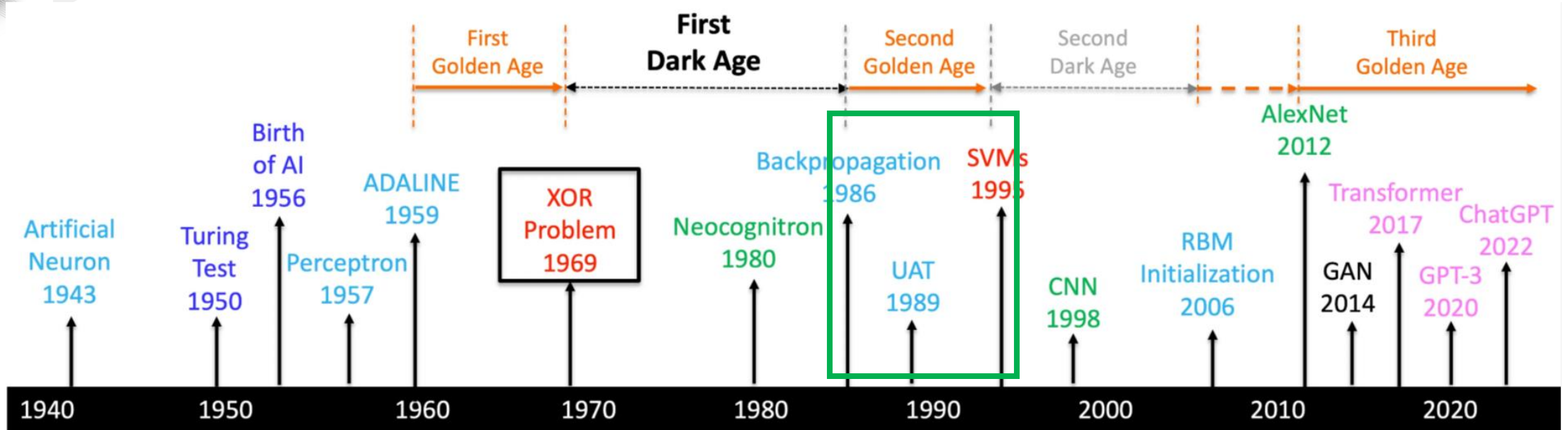


5. Neocognition (1980)

- Là mạng nơ ron nhiều lớp có thứ bậc do Kunihiro Fukushima đề xuất vào 1979.
- Nhận dạng ký tự viết tay Nhật Bản → Mạng tích chập



LỊCH SỬ PHÁT TRIỂN HỌC SÂU

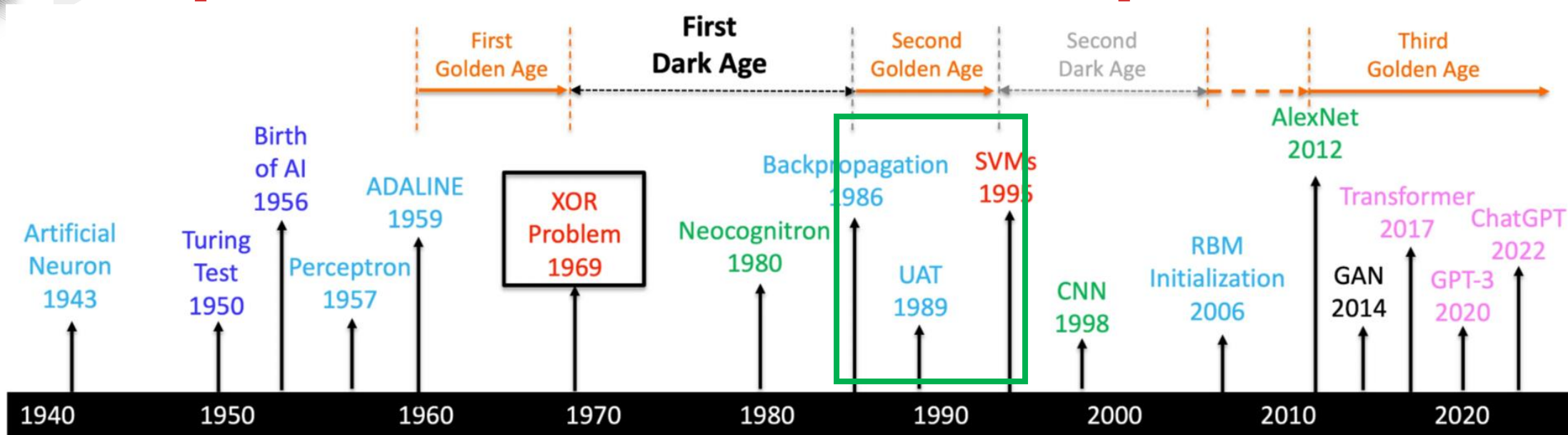


6. Backpropagation (1970s – 1980s)

- Perceptron đa tầng khắc phục được Perceptron.
- Giải thuật Backpropagation huấn luyện mạng phức tạp
- **Seppo Linnainmaa (1970):** đề xuất đạo hàm tự động cho giải thuật backpropagation
- **Paul Werbos (1974):** đề xuất chuỗi quy tắc vi tích phân tính lan truyền ngược lỗi ứng với weight.



LỊCH SỬ PHÁT TRIỂN HỌC SÂU



6. Backpropagation (1986) - David Rumelhart, Geoffrey Hinton, and Ronald Williams

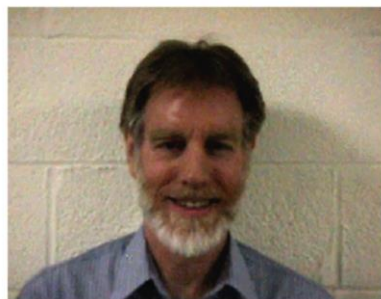
- Trình bày thực tế và ứng dụng mạng nơ ron sâu



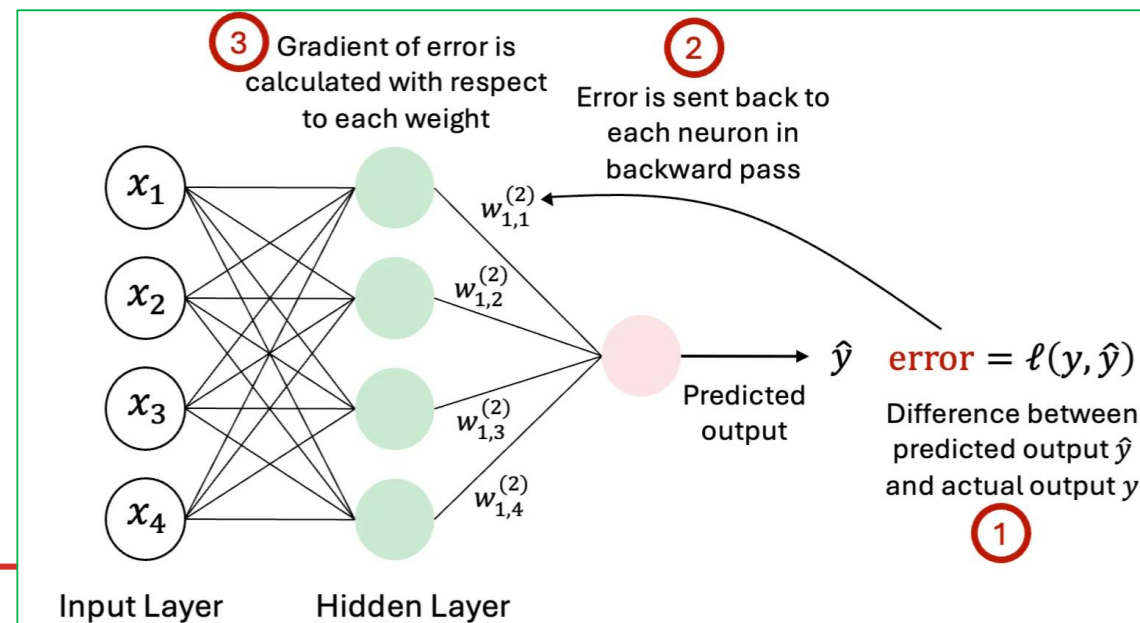
David Rumelhart



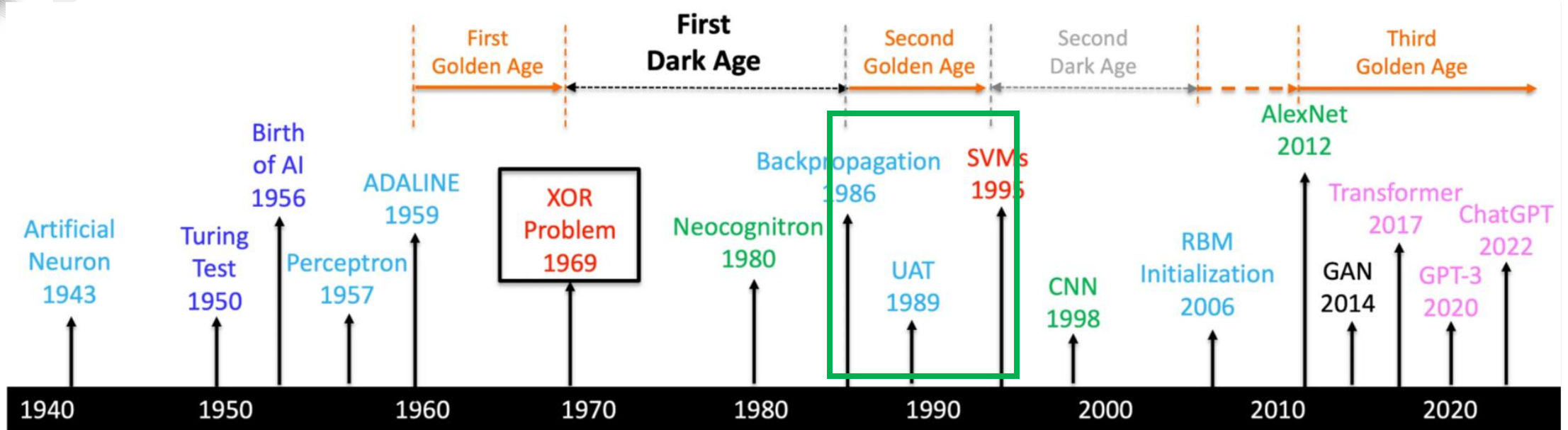
Geoffrey Hinton



Ronald Williams

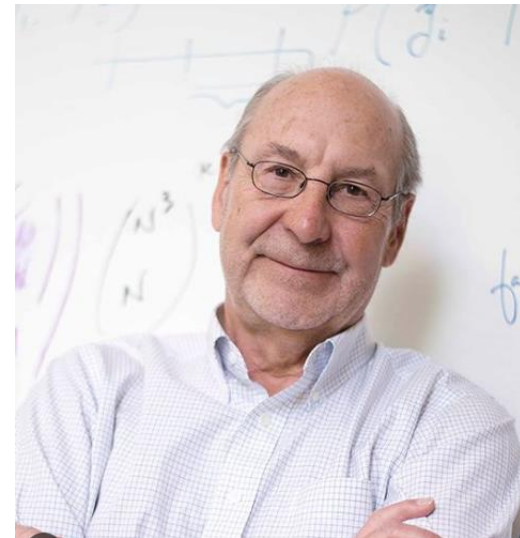
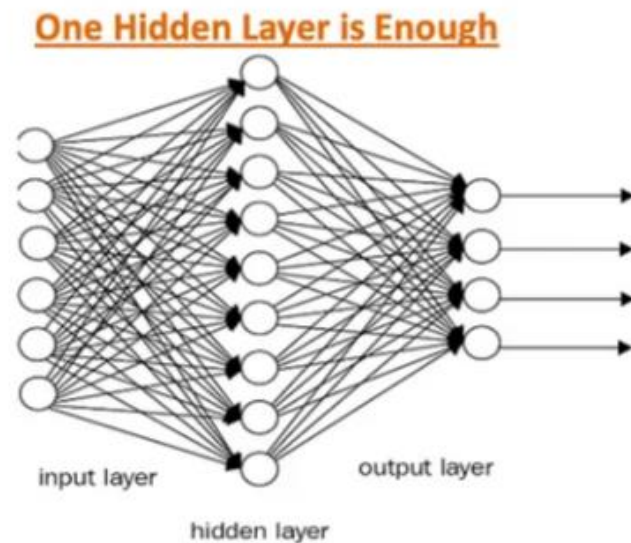


LỊCH SỬ PHÁT TRIỂN HỌC SÂU

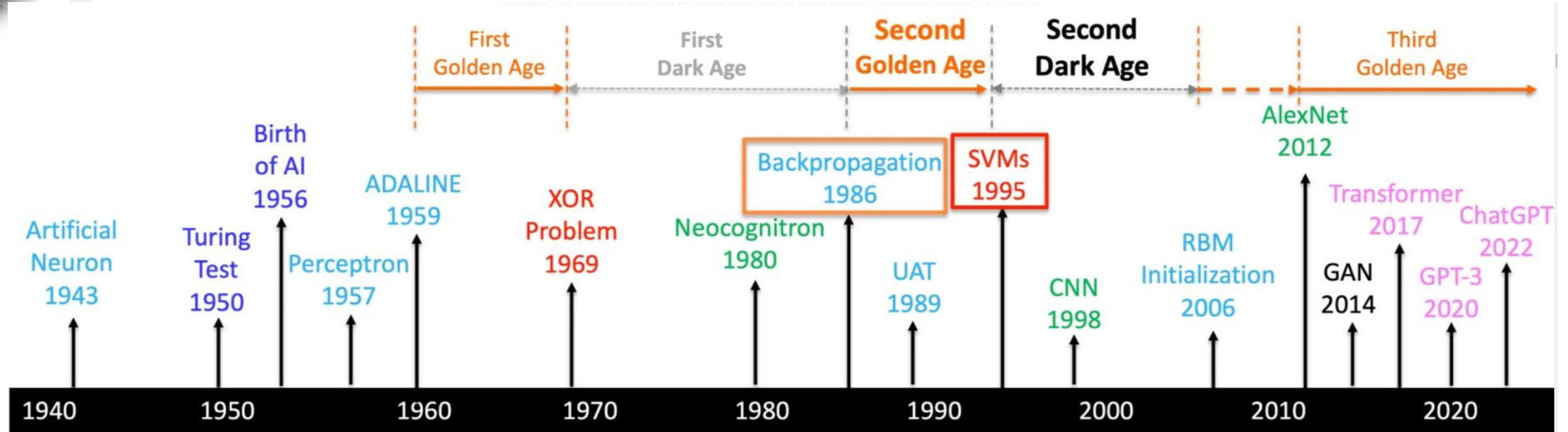


7. Universal Approximation Theorem (1989)

- Do George Cybenko đề xuất.
- Mạng nơ ron có một tầng ẩn duy nhất có thể xấp xỉ hàm liên tục với độ chính xác tùy ý có đủ số nơ ron và sử dụng hàm non linear activation.
- UAT phù hợp nhiều ứng dụng thực tế.



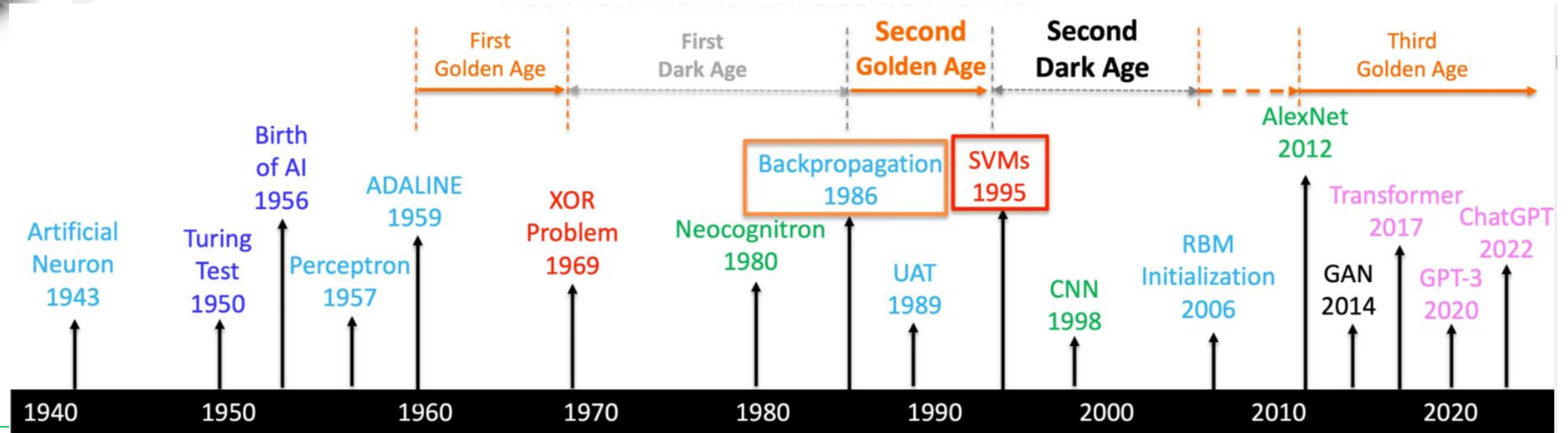
LỊCH SỬ PHÁT TRIỂN HỌC SÂU



8. Second Dark Ages (Early 1990s — Early 2000s)

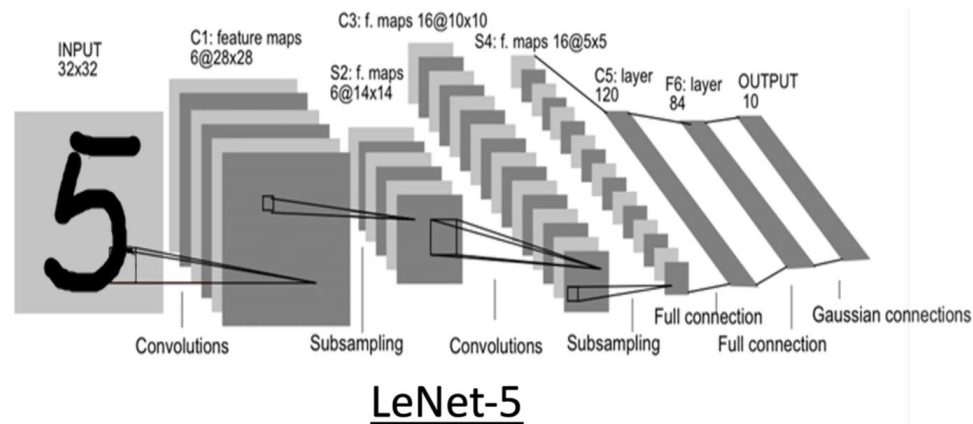
- Sự trỗi dậy của Support Vector Machine (SVM): mô hình toán cho phân lớp và hồi qui
- Giới hạn tính toán: tốn nhiều thời gian và tài nguyên máy tính huấn luyện mạng nơ ron
- Vấn đề overfitting và generalization: Huấn luyện tốt trên tập train nhưng dự báo sai trên tập test.

LỊCH SỬ PHÁT TRIỂN HỌC SÂU

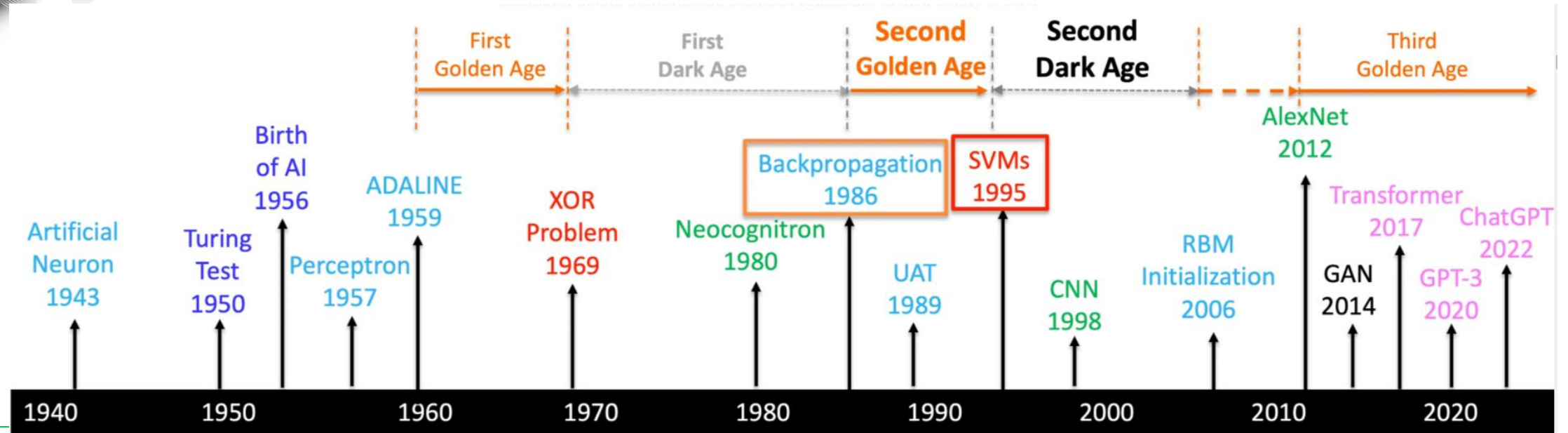


9. Hồi sinh NN (Cuối 2000s - nay) – Mạng Nơ ron chập (1989 – 1998)

- Yann LeCun phát triển Convolutional Neural Network (CNN) cuối 1980s và đầu 1990s với mô hình LeNet-5 nhận dạng chữ viết tay.

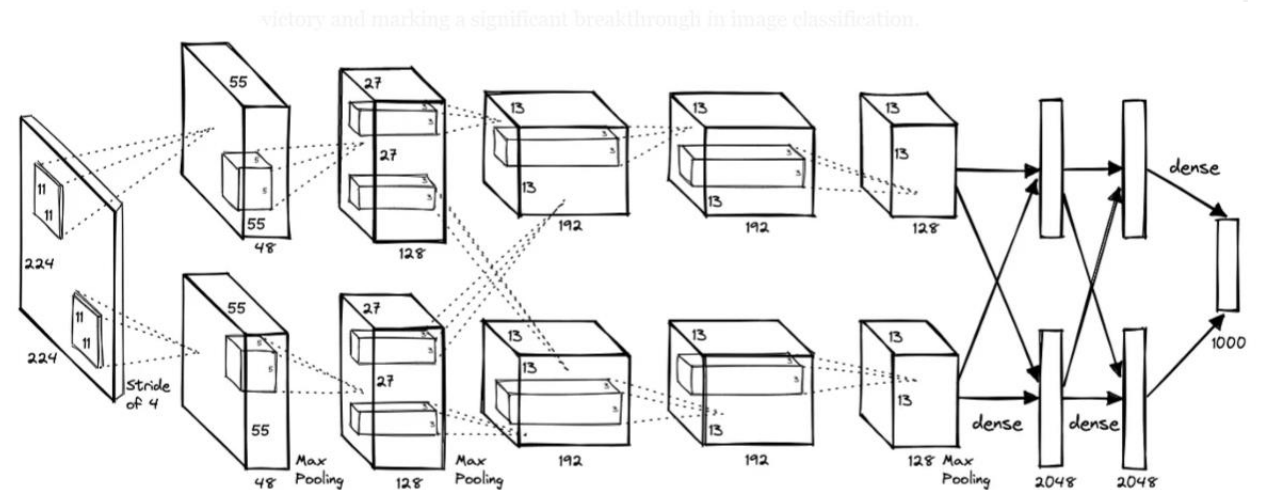


LỊCH SỬ PHÁT TRIỂN HỌC SÂU



10. AlexNet (2012)

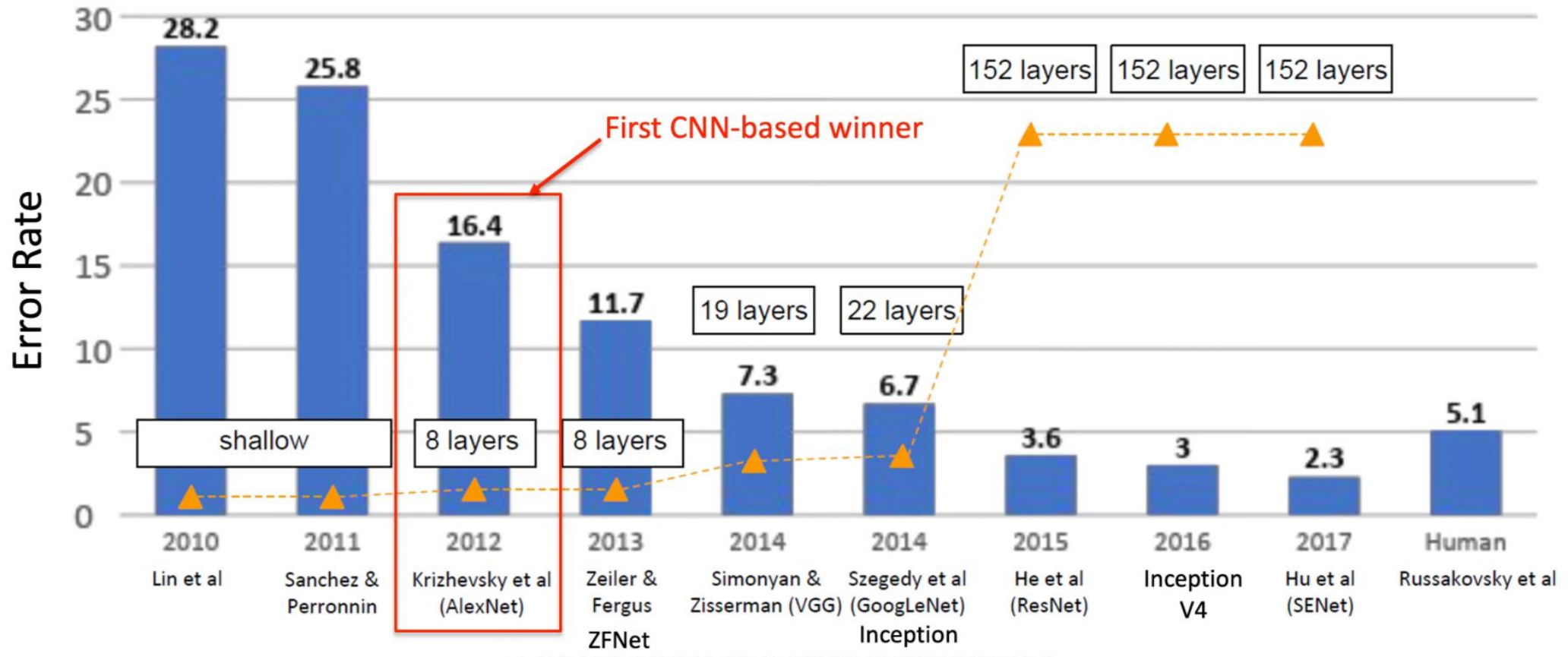
- Giành chiến thắng tại cuộc thi ImageNet Large Scale Visual Recognition Challenge cho bài toán phân lớp (10 triệu ảnh, 1,000 lớp)
- Giới thiệu ReLU (fast training), dropout (giảm overfitting)



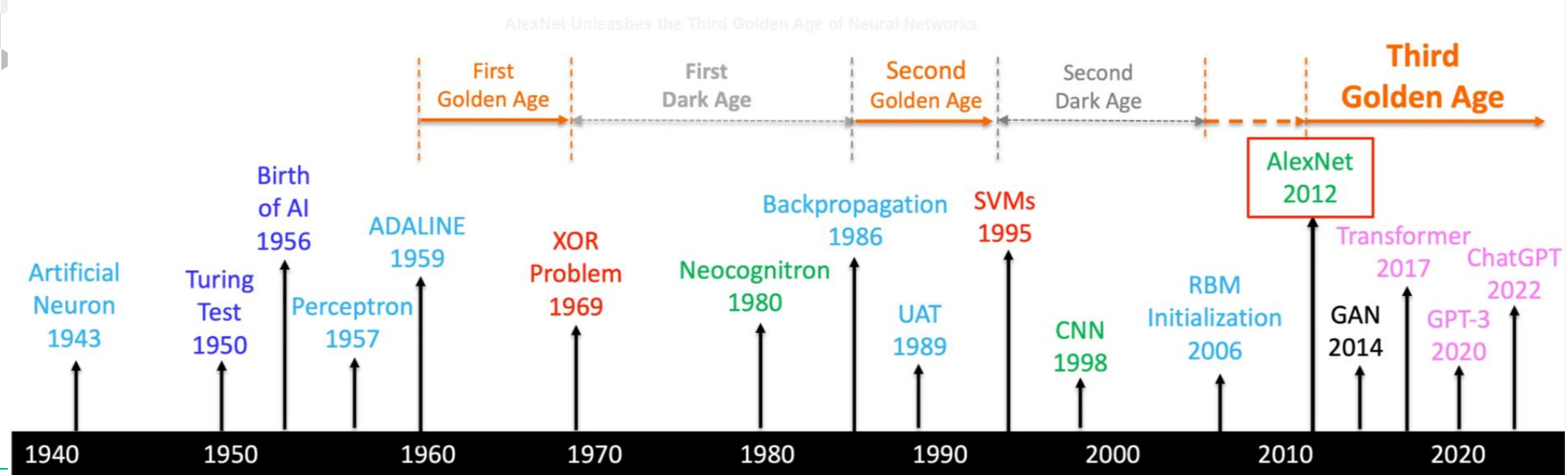
3. Data Augmentation: Enhancements to the training dataset improved

LỊCH SỬ PHÁT TRIỂN HỌC SÂU

Winners of ImageNet Classification Challenge (ILSVRC)

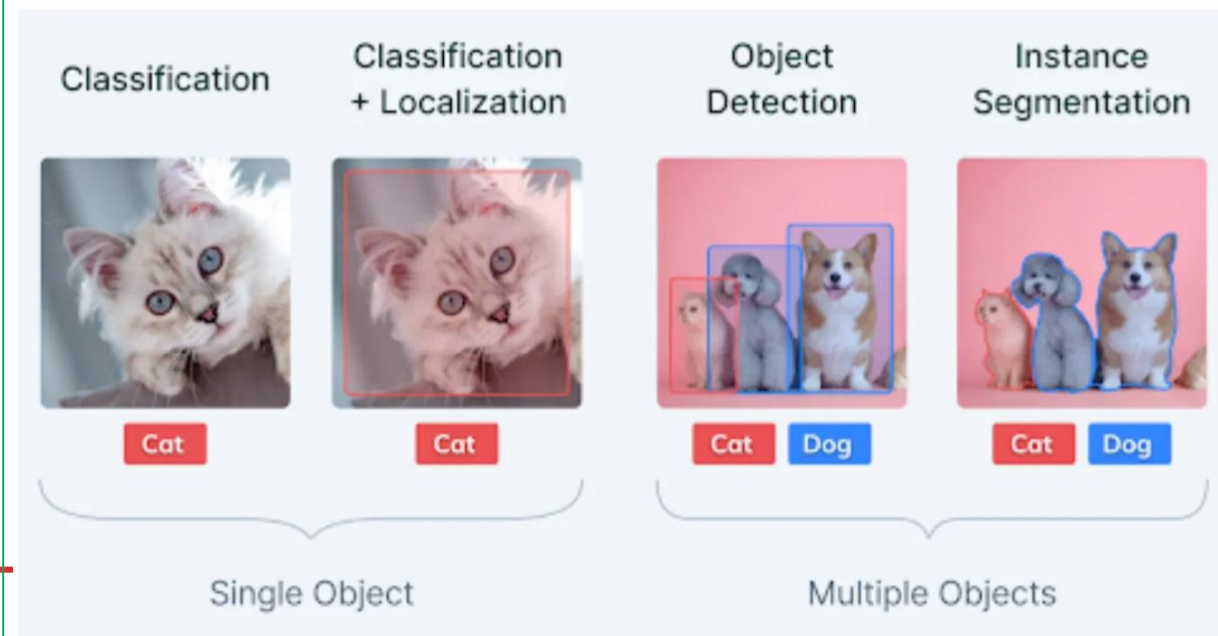


LỊCH SỬ PHÁT TRIỂN HỌC SÂU

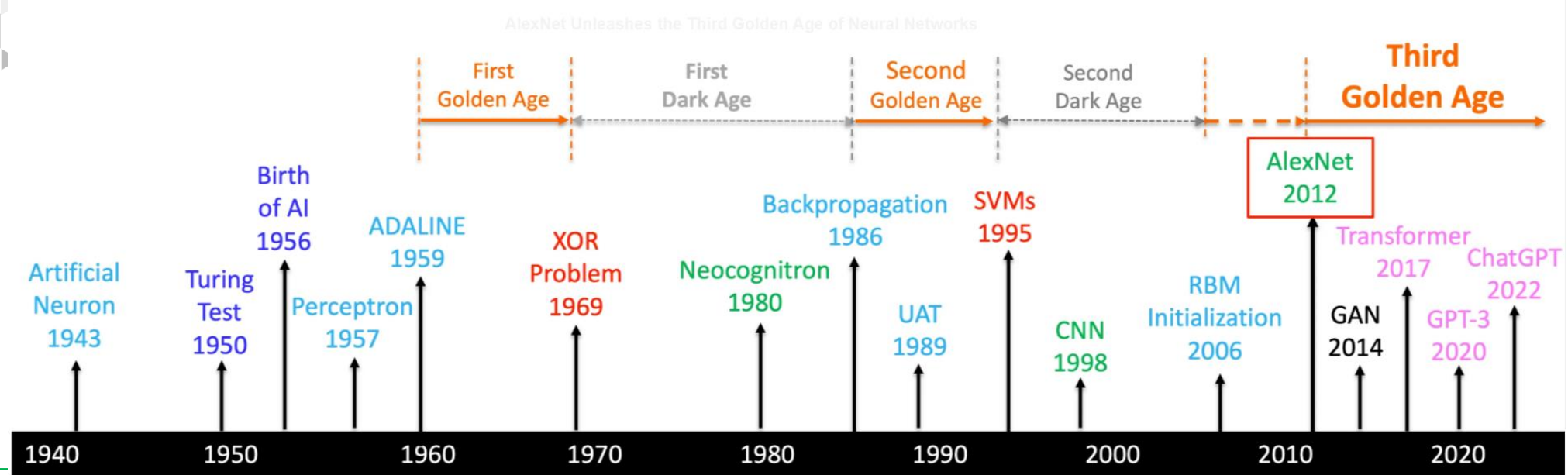


11. Thời kỳ hoàng kim lần 3 (2010s - nay)

- VGGNet (2014), GoogLeNet/Inception (2014), ResNet (2015)
- Ứng dụng CNN trong thị giác máy tính (image classification, object detection, semantic segment)
- Ảnh y khoa: chẩn đoán bệnh truyền nhiễm, phát hiện khối u
- Xe tự hành

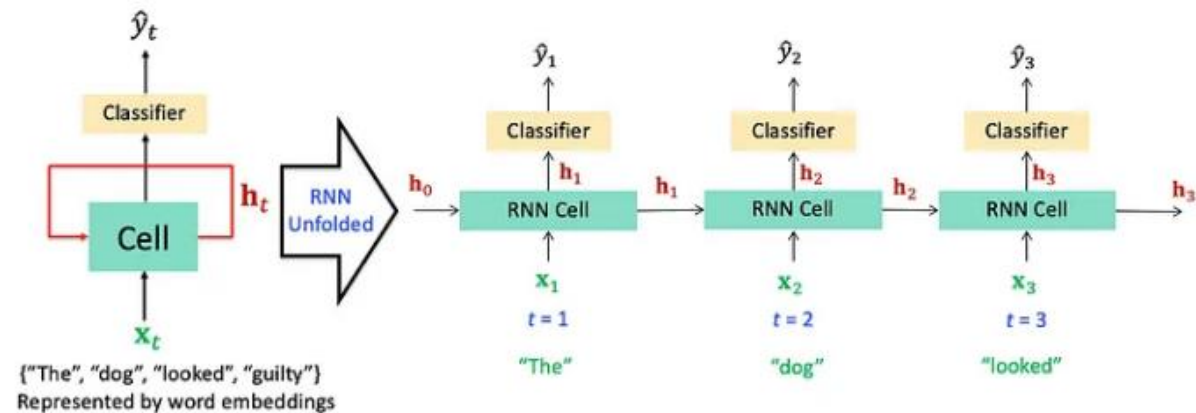


LỊCH SỬ PHÁT TRIỂN HỌC SÂU

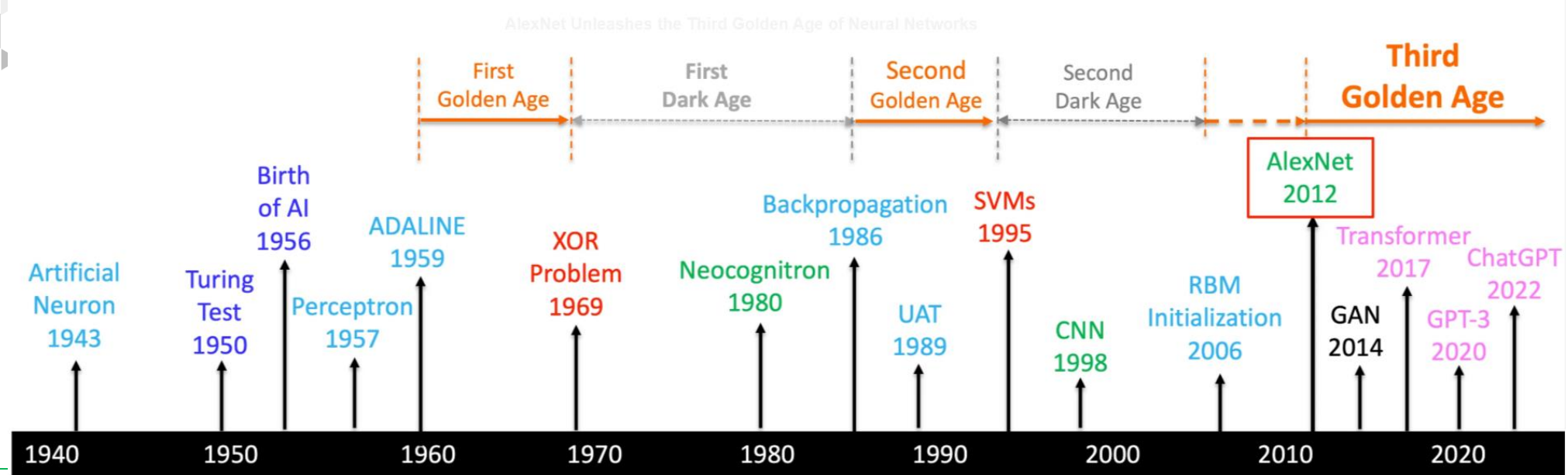


12. Mạng nơ ron hồi quy (Recurrent NN) (1986 - 2017)

- John Hopfield đề xuất 1982
- Michael I. Jordan và Jeffrey L. Elman đề xuất 1980s – 1990s
- RNN gặp vấn đề độ dốc quá nhỏ dẫn đến không học gì. Ngược lại, độ dốc lớn dẫn đến học không ổn định (Vanishing gradient).

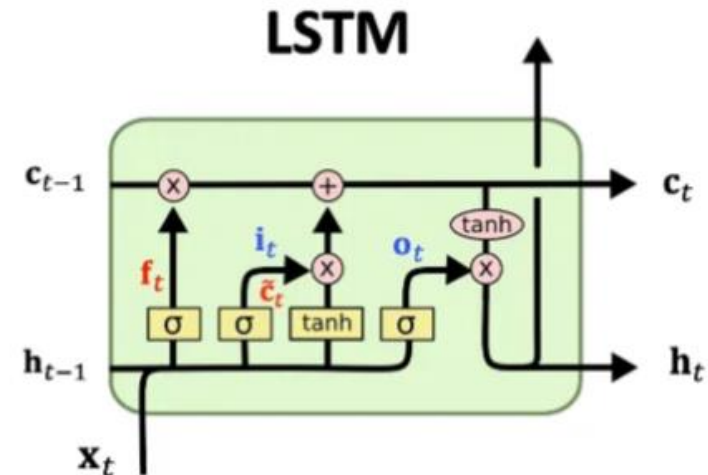


LỊCH SỬ PHÁT TRIỂN HỌC SÂU

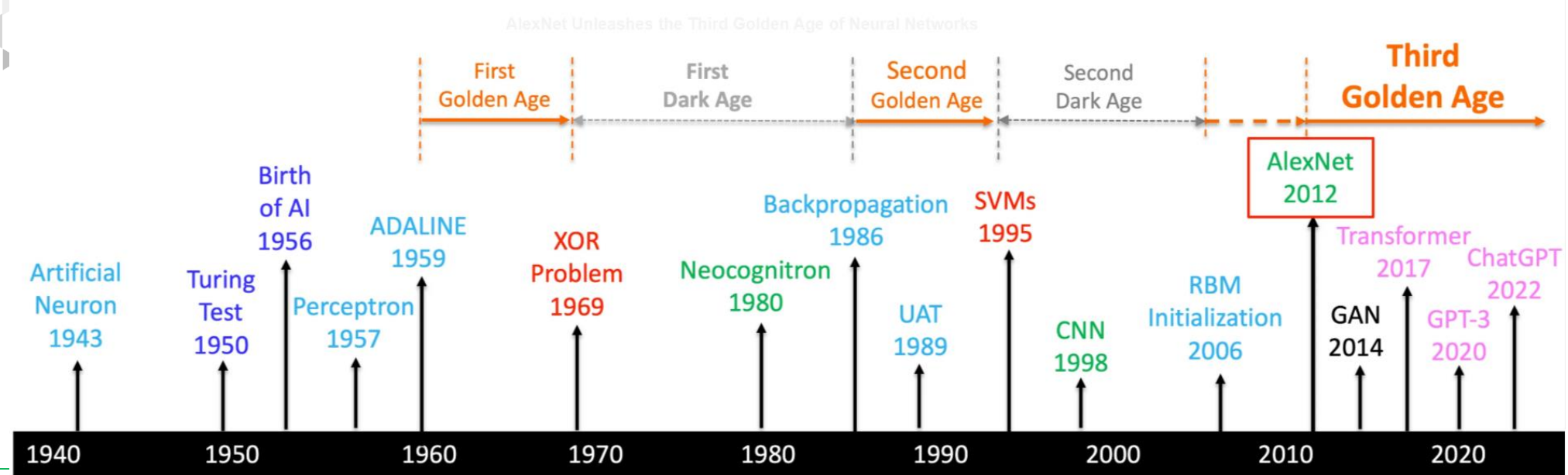


13. LSTM (1997 — 2014)

- Long Short-Term Memory (1997): do Sepp Hochreiter and Jürgen Schmidhuber đề xuất, giải quyết vấn đề vanishing gradient
- Sử dụng **Cell state** (store long-term information), **Hidden state** (to carry the short-term output for the current time step), and **Three gates** (Input, Forget, Output gates)

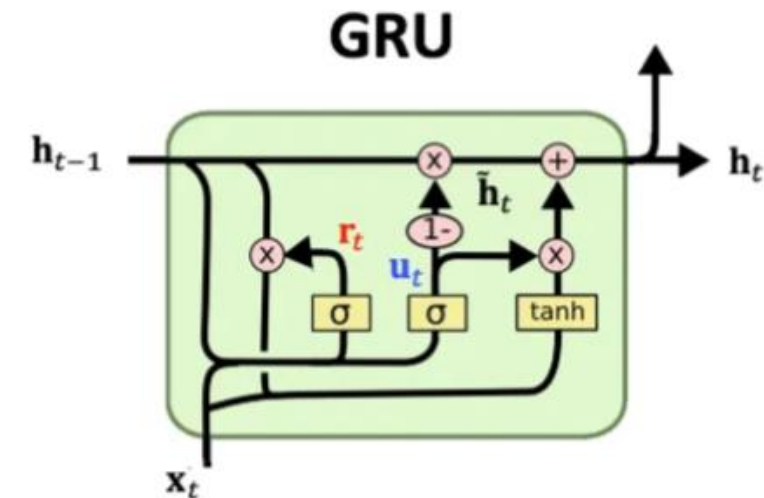


LỊCH SỬ PHÁT TRIỂN HỌC SÂU

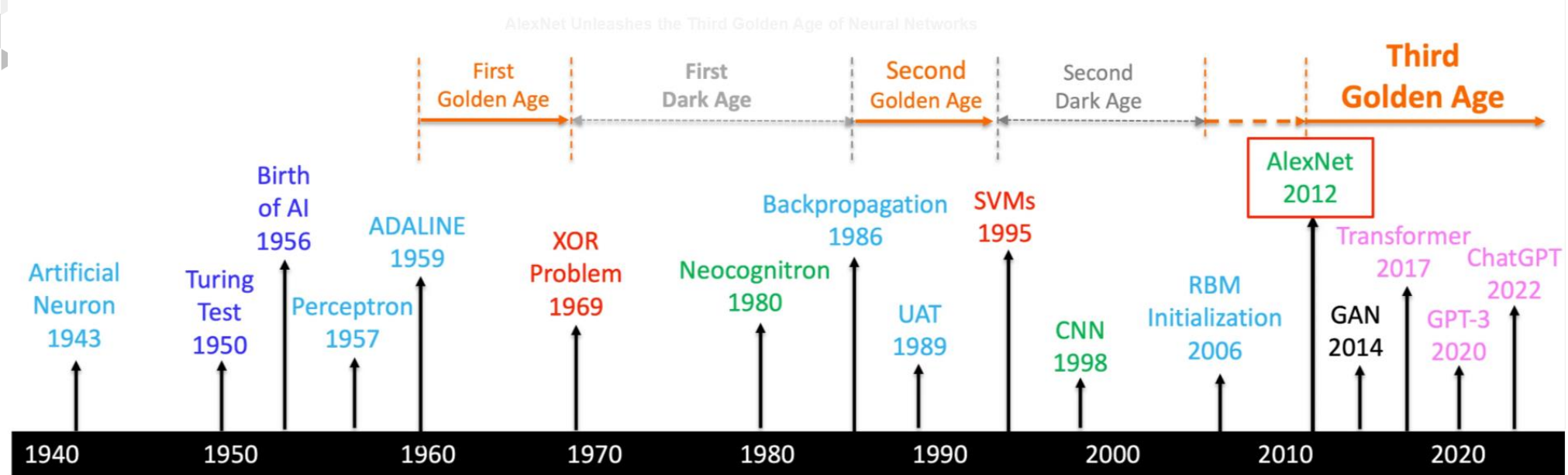


14. GRU (2014)

- **Gated Recurrent Units (2014):** do Kyunghyun Cho đề xuất, rút gọn hơn LSTM.
- Sử dụng 2 gates và 1 states. Forget và Input gates (LSTM) hợp lại thành Update gate.
- Output gate (LSTM) thay bằng Reset gate
- Ít tham số hơn LSTM nên huấn luyện nhanh hơn

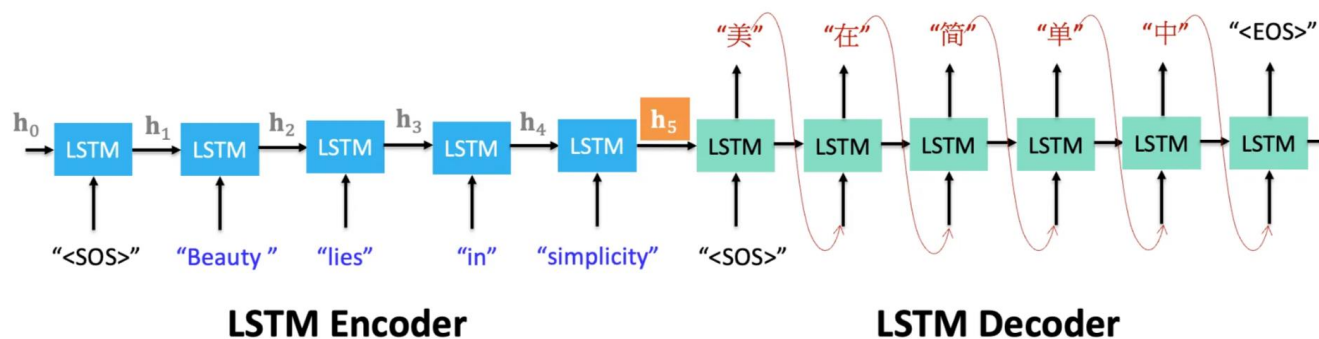


LỊCH SỬ PHÁT TRIỂN HỌC SÂU

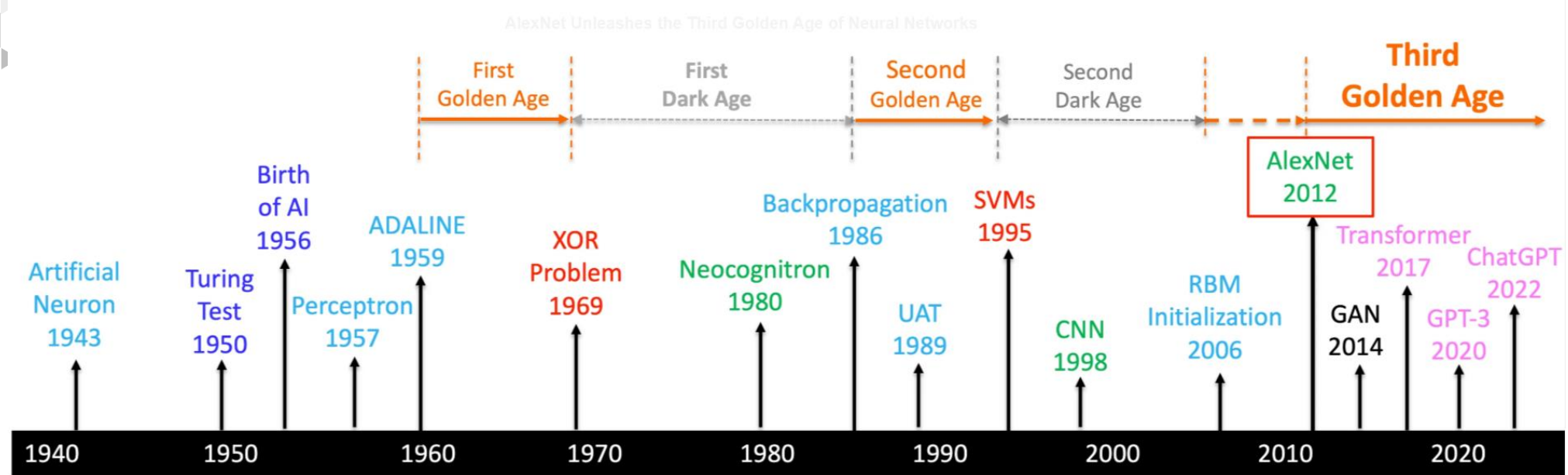


15. Sequence to sequence (seq2seq) (2014)

- Do Ilya Sutskever đề xuất
- Sử dụng kiến trúc encoder và decoder và ứng dụng trong dịch máy, nhận dạng giọng nói, tóm tắt văn bản

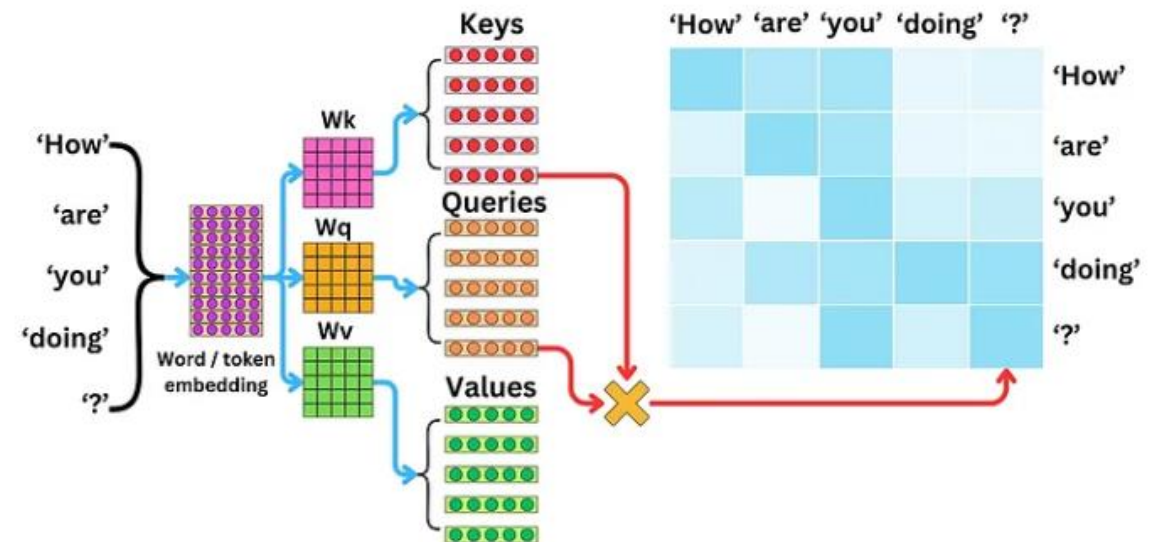


LỊCH SỬ PHÁT TRIỂN HỌC SÂU



16. Transformer (2017)

- Do Vaswani đề xuất
- Sử dụng self-attention, cho phép xử lý song song và hiểu được chuỗi dài

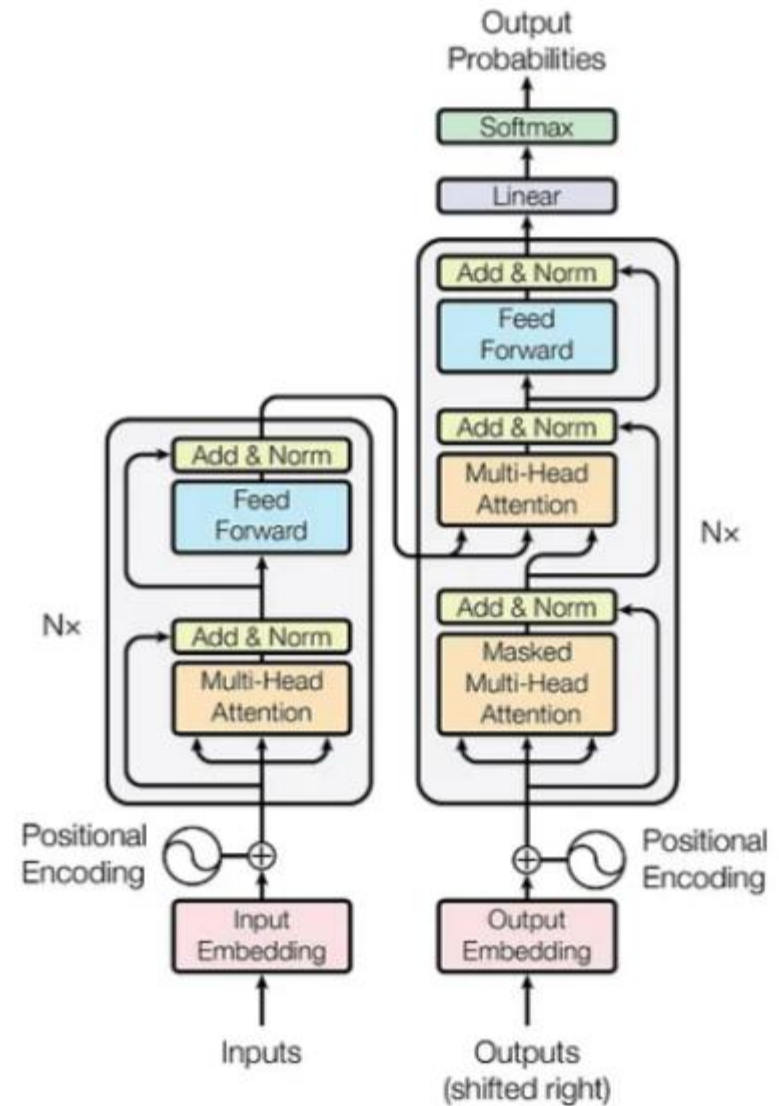


Self-Attention mechanism (Source)

LỊCH SỬ PHÁT TRIỂN HỌC SÂU

16. Transformer (2017)

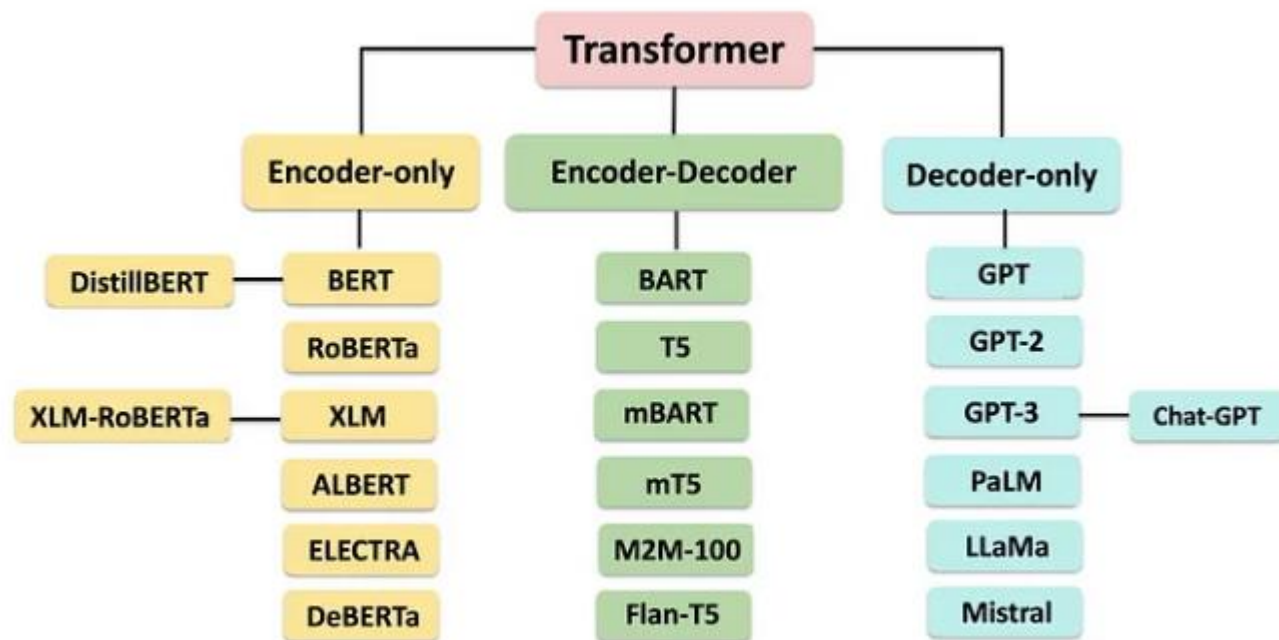
- **Self-Attention:** Cho phép mỗi vị trí trong chuỗi chú ý đến tất cả vị trí khác, hiểu được ngữ cảnh tốt hơn RNN/LSTM
- **Parallelization:** Xử lý tất cả dữ liệu cung cấp đồng thời, ngược hoàn toàn với RNN.
- **Encoder-Decoder Structure:** cả encoder và decoder sử dụng self attention, feed forward neural và position encoding để duy trì trật tự các từ trong chuỗi.



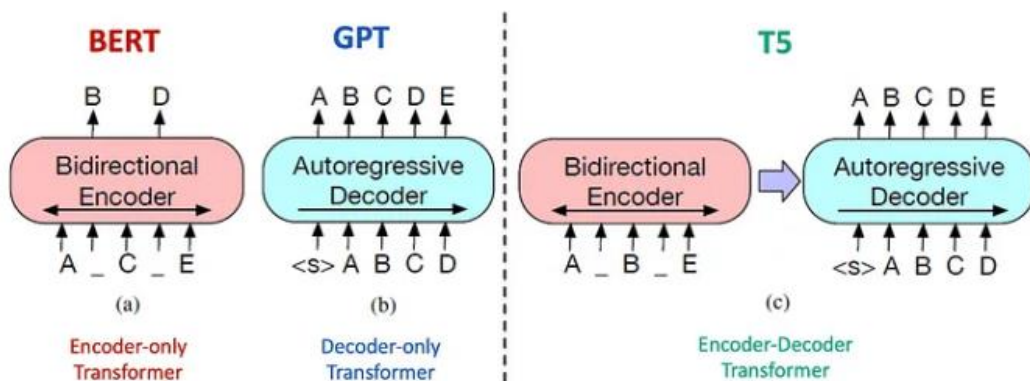
Original Transformer Architecture with Encoder-Decoder structure and Multi-Head Attention.

CÁC MÔ HÌNH DỰA TRÊN TRANSFORMER

- **BERT (2018):** Bidirectional Encoder Representations from Transformers, một **encoder-only transformer**, cách mạng hóa NLP với tiền huấn luyện trên ngôn ngữ mặt nạ và dự báo câu tiếp theo.
- **T5 (2019):** Text-to-Text Transfer Transformer, một **encoder-decoder transformer**, nhóm các tác vụ NLP vào hình thức text-to-text, kiểm trúc mô hình đơn giản.



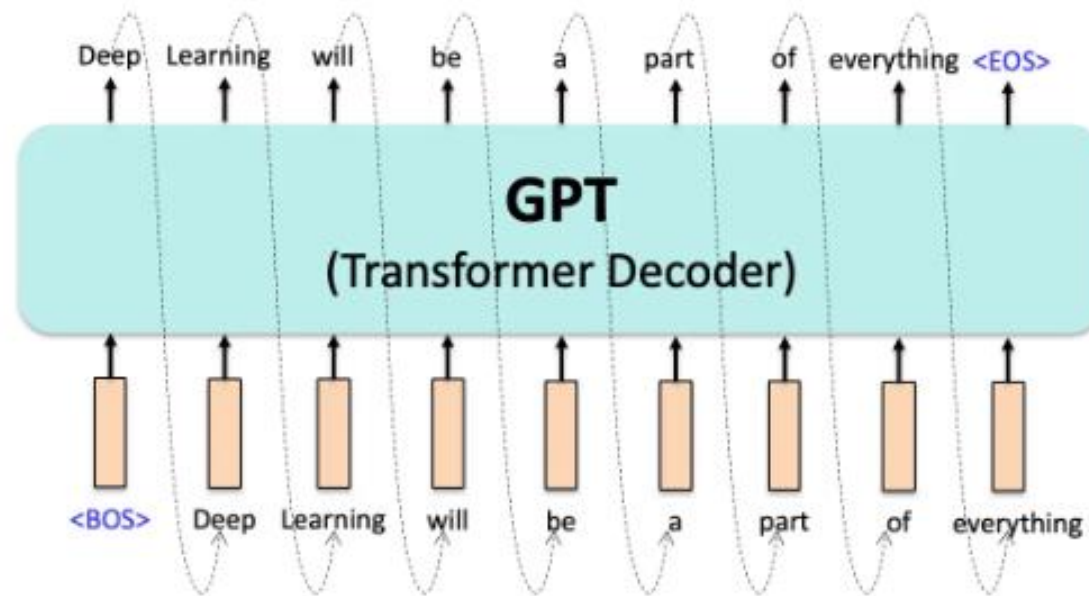
Transformer-based Language Models



BERT vs GTP vs T5

TỔNG QUAN CÁC DÒNG OPENAI GPT

- OpenAI's Generative Pre-trained Transformer (GPT) có những tiến bộ đáng kể trong xử lý ngôn ngữ tự nhiên (NLP) vào 2018.



The Autoregressive Language Model architecture of GPT is designed to predict the next token in a sequence based on the previous tokens inputted.

TỔNG QUAN CÁC DÒNG OPENAI GPT

- **GPT (2018):** Mô hình GPT đầu tiên được giới thiệu vào năm 2018, là một autoregressive **decoder-only transformer** với **117 million parameters**.
 - Được thiết kế để dự báo token tiếp theo (**sub-word**) trong một chuỗi, minh họa khả năng hiểu và sinh từ như con người.
 - Đặt nền tảng cho những phát triển tiếp theo trong các mô hình ngôn ngữ tạo sinh, thể hiện tiềm năng của việc học không giám sát từ tập văn bản lớn
- **GPT-2 (2019):** tham số mô hình 1.5 tỷ, thực hiện tác vụ zero-shot mà không cần huấn luyện trên dữ liệu miền.
 - Khả năng sinh nội dung chưa mạch lạc và đôi khi liên quan đến vấn đề đạo đức

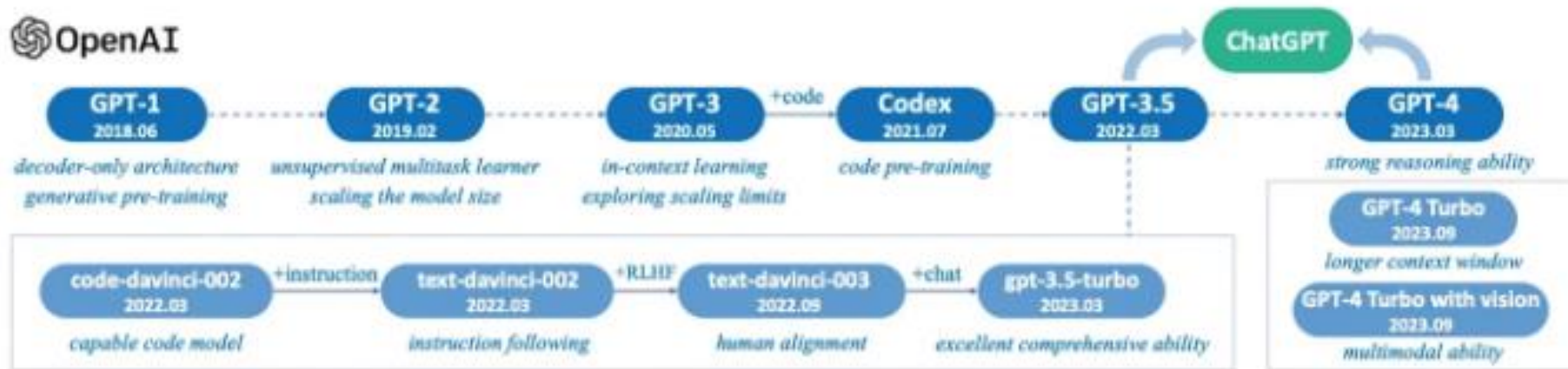
TỔNG QUAN CÁC DÒNG OPENAI GPT

- **GPT-3 (2020): có 175 tỷ tham số.**
 - Có khả năng học few-shot với ít mẫu cung cấp trong quá trình prompt.
 - Có khả năng sinh văn bản tựa người.
 - Công cụ tạo nội dung, viết code, trao đổi.
- **ChatGPT (2022): phiên bản 3.5**
 - Tối ưu hóa nhiều lượt trao đổi với Reinforcement Learning from Human Feedback.
 - Xử lý câu hỏi tiếp theo và duy trì ngữ cảnh.
 - Sắp xếp câu trả lời theo ý định của người dùng qua việc điều chỉnh hướng dẫn và dữ liệu tùy chọn

TỔNG QUAN CÁC DÒNG OPENAI GPT

- GPT-4 (2023):

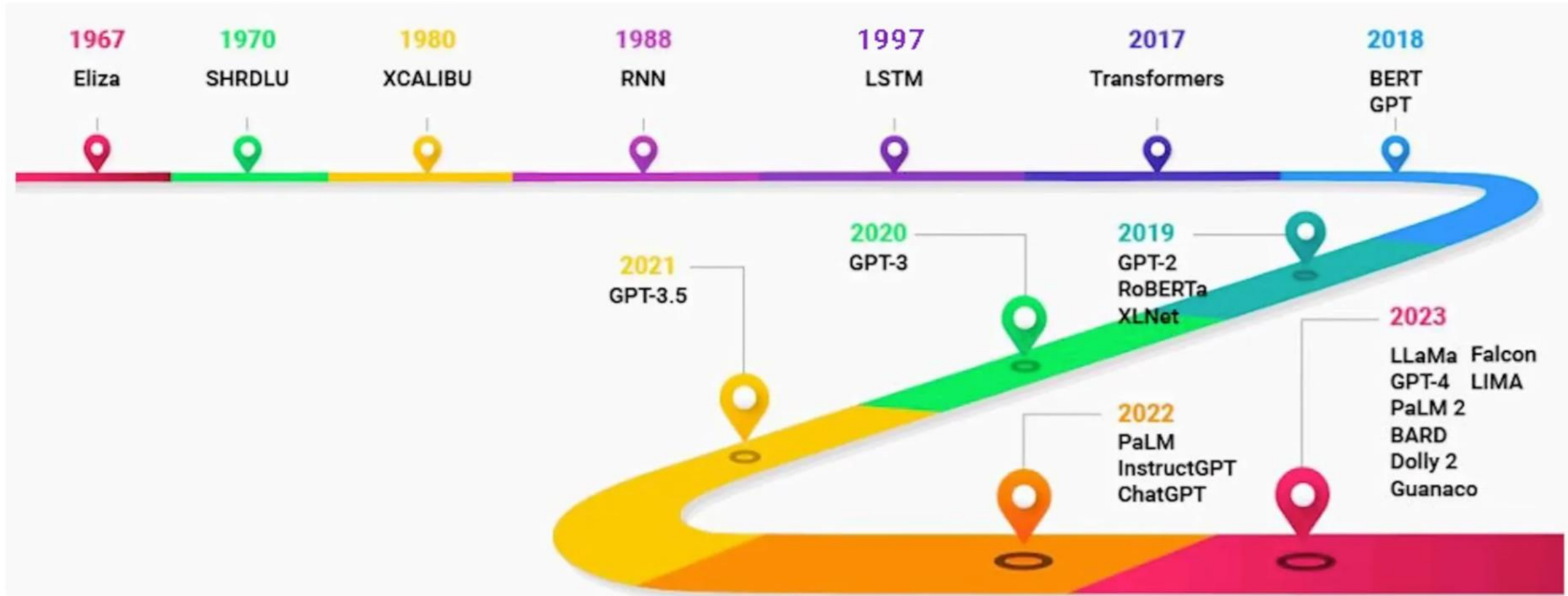
- Tập trung vào suy luận và hiểu ngữ cảnh phức tạp



<https://klu.ai/glossary/large-language-model>

CÁC MÔ HÌNH LLM KHÁC

Evolution of Large Language Models



CÁC MÔ HÌNH LLM MULTIMODAL

- GPT-4V (2023) và GPT-4o (2024)

- GPT-4V (2023) : tích hợp khả năng đa dạng vấn đề ngoài việc sinh văn bản

GPT-4 visual input example, Extreme Ironing:

User What is unusual about this image?



Source: <https://www.barnorama.com/wp-content/uploads/2016/12/03-Confusing-Pictures.jpg>

GPT-4 The unusual thing about this image is that a man is ironing clothes on an ironing board attached to the roof of a moving taxi.



CÁC MÔ HÌNH LLM MULTIMODAL

- GPT-4V (2023) và GPT-4o (2024)

- **GPT-4o (2024)** : một sự cải tiến của GPT-4V, sinh ảnh từ prompt, và tinh chỉnh khả năng lập luận
- Nâng cấp việc huấn luyện dữ liệu có chuẩn mực đạo đức nhằm đảm bảo câu trả lời chính xác và có giá trị cho con người hơn.
- Cải thiện tốc độ và hiệu suất
- Có khả năng truy cập internet để thu thập thông tin trực tuyến
- Cho phép người dùng kiểm tra thông tin thời gian thực



NỘI DUNG

01. GIỚI THIỆU HỌC SÂU

02. KHÁI NIỆM CƠ BẢN HỌC SÂU

03. MẠNG NEURAL

04. MẠNG NEURAL TÍCH CHẬP

05. MẠNG NEURAL HỒI QUY

06. MẠNG ENCODER-DECODER

GIỚI THIỆU KHÁI NIỆM HỌC SÂU CƠ BẢN

- **Mạng học sâu** là mô hình toán học lấy cảm hứng từ cấu trúc não người.
- Mô hình toán học là một công thức mô tả quan hệ giữa biến đầu vào và biến đầu ra.
- VD: mô hình toán của đường thẳng

$$y = mx + c$$

Trong đó,

y: biến đầu ra

x: biến đầu vào

c : là biên cho giá trị output khi giá trị input là 0 (intercept)

m : là độ dốc của đường thẳng, đo lường tầm quan trọng của input với output

GIỚI THIỆU KHÁI NIỆM HỌC SÂU CƠ BẢN

Ví dụ: Giả định rằng thu nhập hằng năm ảnh hưởng đến hanh phúc của một người. Mô hình toán cho giả định này

$$\text{happiness} = m * \text{income} + c$$

Trong đó,

happiness: biến đầu ra

income: biến đầu vào

m, c : là các tham số mô hình điều chỉnh quan hệ giữa income và happiness

GIỚI THIỆU KHÁI NIỆM HỌC SÂU CƠ BẢN

$C = 1, m = 0.08, \text{income} = 0,$

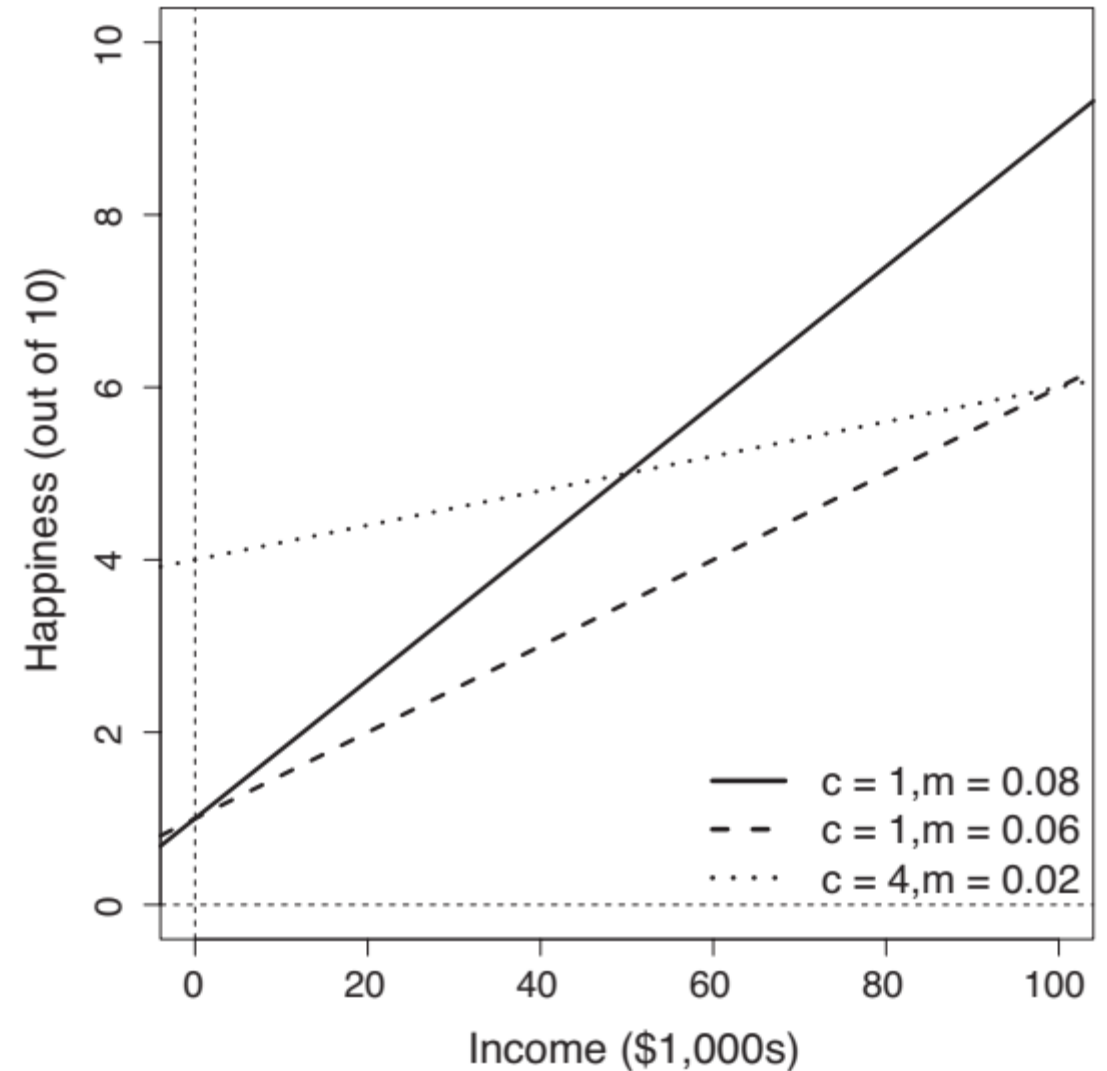
$\rightarrow \text{happiness} = 0.08 * 0 + 1 = 1$

$C = 1, m = 0.06, \text{income} = 0$

$\rightarrow \text{happiness} = 0.06 * 0 + 1 = 1$

$C = 1, m = 0.02, \text{income} = 0$

$\rightarrow \text{happiness} = 0.02 * 0 + 4 = 4$



MÔ HÌNH TUYẾN TÍNH VỚI NHIỀU INPUT

Ví dụ: Xem xét hồ sơ vay cấp tín dụng của một cá nhân qua 2 yếu tố thu nhập hàng năm và nợ vay hiện tại. Mô hình toán cho giả định này

$$\text{Solvency} = (\text{weight for income} * \text{income}) + (\text{weight for debt}) * \text{debt} + \text{intercept}$$

Viết lại

$$y = (\text{input}_1 * \text{weight}_1) + (\text{input}_2 * \text{weight}_2) + c$$

Viết gọn hơn

$$y = \sum_{i=1}^n (\text{input}_i \times \text{weight}_i) + c$$

→ Còn gọi là weighted sum

THIẾT LẬP THAM SỐ CHO MÔ HÌNH TUYẾN TÍNH

- Ví dụ: Xem xét hồ sơ vay cấp tín dụng của một cá nhân qua 2 yếu tố thu nhập hàng năm và nợ vay hiện tại. Mô hình toán cho giả định này

$$\text{Solvency} = (\text{weight for income} * \text{income}) + (\text{weight for debt}) * \text{debt} + \text{intercept}$$

- Giả định giá trị income ảnh hưởng đến quyết định hồ sơ vay tín dụng
- Thiết lập tham số $\text{weight for income} > \text{weight for debt}$ (tư vấn chuyên gia)

$$\text{solvency} = (\text{income} \times 3) + (\text{debt} \times 1)$$

- Tham số này gây tranh cãi: ý kiến chuyên gia

→ Dùng học máy quyết định tham số w_i

HỌC THAM SỐ TỪ MÔ HÌNH TUYẾN TÍNH

- Dùng giải thuật học máy gradient descent để học tham số w_1 và w_2 thông qua dataset cung cấp

ID	Annual income	Current debt	Credit solvency
1	\$150	-\$100	100
2	\$250	-\$300	-50
3	\$450	-\$250	400
4	\$200	-\$350	-300

HỌC THAM SỐ TỪ MÔ HÌNH TUYẾN TÍNH

B1. Khởi động $w_1 = 3$; $w_2 = 1$ (bất kỳ)

B2. Thay vào mẫu 1 ($x_1 = 150$; $x_2 = -100$): $\hat{y} = 3 * 150 + 1 * (-100) = 350 > 100$ (y)

B3. Nếu $\text{error} = y - \hat{y} = 0$ thì không nên thay đổi weight

- Nếu $\text{error} = y - \hat{y} > 0$ thì dự báo đầu ra quá nhỏ $\rightarrow$ cần tăng trọng số weight có input > 0 và giảm trọng số weight có input < 0
- Nếu $\text{error} = y - \hat{y} < 0$ ($100 - 350 < 0$) thì dự báo đầu ra quá lớn $\rightarrow$ cần giảm trọng số weight có input > 0 và tăng trọng số weight có input < 0

$\rightarrow w_1 = 2$; $w_2 = 2 \rightarrow \hat{y} = 2 * 150 + 2 * (-100) = 100 = (y)$

$\rightarrow$ Thực hiện nhiều lần (EPOCH) để tìm bộ trọng số (w_1, w_2) cho toàn bộ dataset.

BÀI TẬP HỌC THAM SỐ TỪ MÔ HÌNH TUYẾN TÍNH

- Hãy tìm bộ trọng số cho bảng sau bằng giải thuật học vừa trình bày

ID	Annual income	Current debt	Credit solvency
1	\$150	-\$100	100
2	\$250	-\$300	-50
3	\$450	-\$250	400
4	\$200	-\$350	-300

MÔ HÌNH KẾT HỢP

Ví dụ: Xem xét hồ sơ vay cấp tín dụng của một cá nhân qua 2 yếu tố thu nhập hàng năm và nợ vay hiện tại. Mô hình toán cho giả định này

$$\text{Solvency} = (\text{weight for income} * \text{income}) + (\text{weight for debt}) * \text{debt} + \text{intercept}$$

*** Ngoài cấp tín dụng, ta cần phải cấp quyền cho vay một hồ sơ hay không

- Ta sử dụng Solvency như một input cho mô hình ra quyết định cho hồ sơ vay hay không

Điều kiện cho vay (Decision Rule):

Cá nhân có số tín dụng > 200 sẽ được duyệt hồ sơ vay.

loan decision

$$= \text{decision rule}(\text{solvency} = (\text{income} \times 2) + (\text{debt} \times 2))$$

MÔ HÌNH KẾT HỢP

Ví dụ: Xem xét hồ sơ vay cấp tín dụng của một cá nhân qua 2 yếu tố thu nhập hàng năm và nợ vay hiện tại. Mô hình toán cho giả định này

$$\text{Solvency} = (\text{weight for income} * \text{income}) + (\text{weight for debt}) * \text{debt} + \text{intercept}$$

Điều kiện cho vay (Decision Rule):

Cá nhân có số tín dụng > 200 sẽ được duyệt hồ sơ vay.

loan decision

$$= \text{decision rule}(\text{solvency} = (\text{income} \times 2) + (\text{debt} \times 2))$$

$$= \text{decision rule}(\text{solvency} = (150 \times 2) + (-100 \times 2))$$

$$= \text{decision rule}(\text{solvency} = 100)$$

$$= \text{reject}$$

BÀI TẬP

1. Xác định các tham số tốt nhất và số epochs của mô hình dự báo điểm cuối kỳ của sinh viên 13: {5, 1} với bảng dữ liệu sau:

	Study Hours	Prep Exams	Final Exam Score
Student 1	3	2	76
Student 2	7	6	88
Student 3	16	5	96
Student 4	14	2	90
Student 5	12	7	98
Student 6	7	4	80
Student 7	4	4	86
Student 8	19	2	89
Student 9	4	8	68
Student 10	8	4	75
Student 11	8	1	72
Student 12	3	3	76

BÀI TẬP

2. Xác định các tham số tốt nhất và số epochs của mô hình dự báo Product Sales tuần 16: {550, 180, 32} với bảng dữ liệu sau:

Week	Product Sales (Y) - Unit	Operational Cost (X1) - USD	Input Price (X2) - USD	Marketing Staff (X3)
1	360	471.4	235.7	8
2	720	514.3	257.1	20
3	1140	612.9	306.4	22
4	1500	642.9	214.3	24
5	1800	651.4	217.1	30
6	2160	694.3	231.4	36
7	2280	771.4	257.1	36
8	2580	780.0	217.1	38
9	2640	771.4	231.4	36
10	2940	750.0	217.1	38
11	3000	835.7	231.4	50
12	3120	840.0	217.1	54
13	3240	784.3	231.4	58
14	2460	814.3	203.6	50
15	2100	642.9	160.7	48



NỘI DUNG

01. GIỚI THIỆU HỌC SÂU

02. KHÁI NIỆM CƠ BẢN HỌC SÂU

03. MẠNG NEURAL

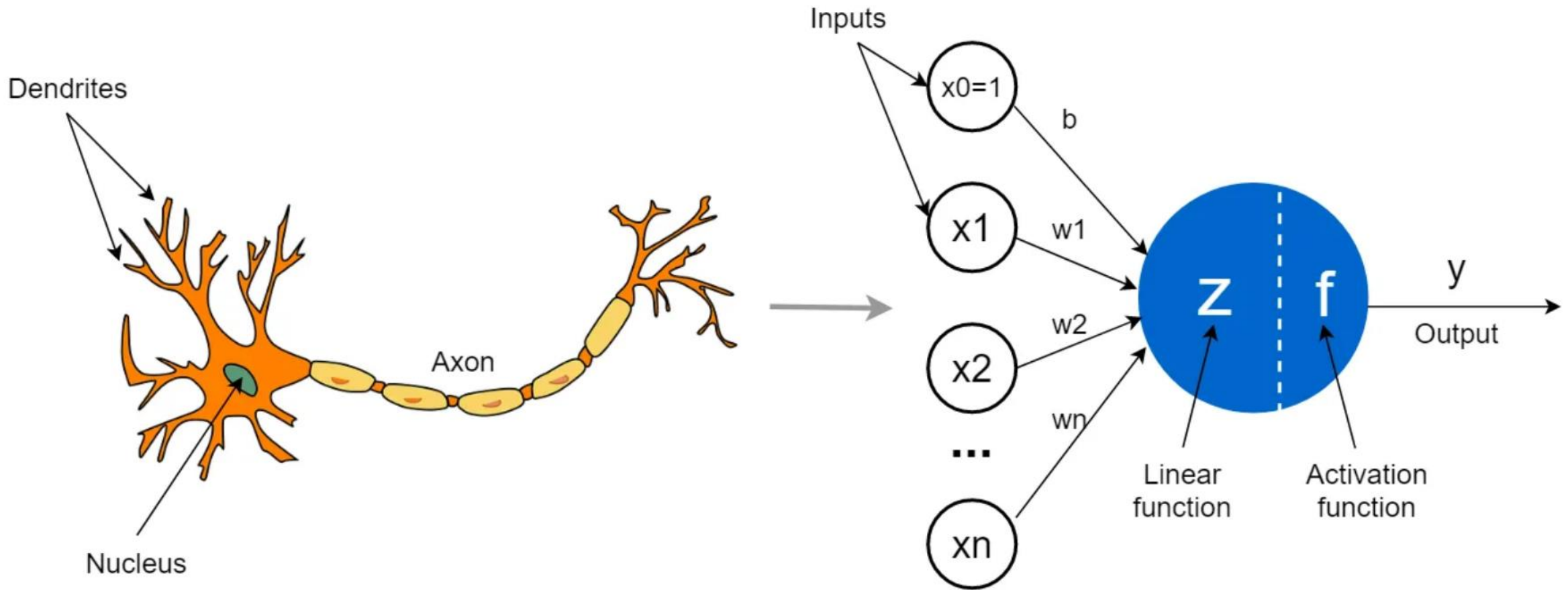
04. MẠNG NEURAL TÍCH CHẬP

05. MẠNG NEURAL HỒI QUY

06. MẠNG ENCODER-DECODER

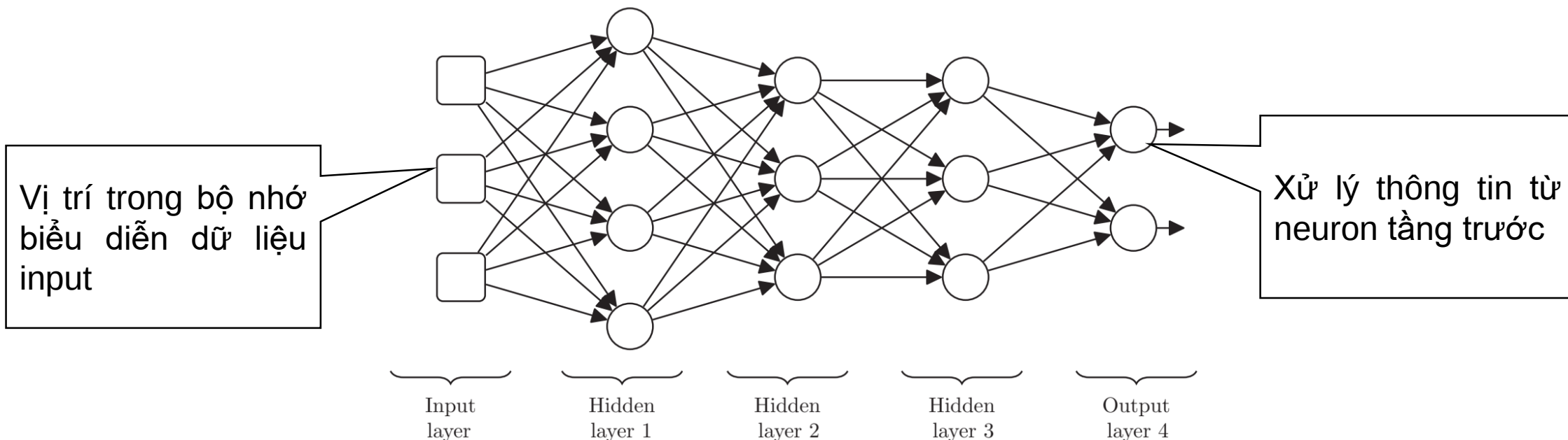
GIỚI THIỆU NEURAL NETWORK

- Mạng nơ-ron là mô hình toán học lấy cảm hứng từ cấu trúc não người.



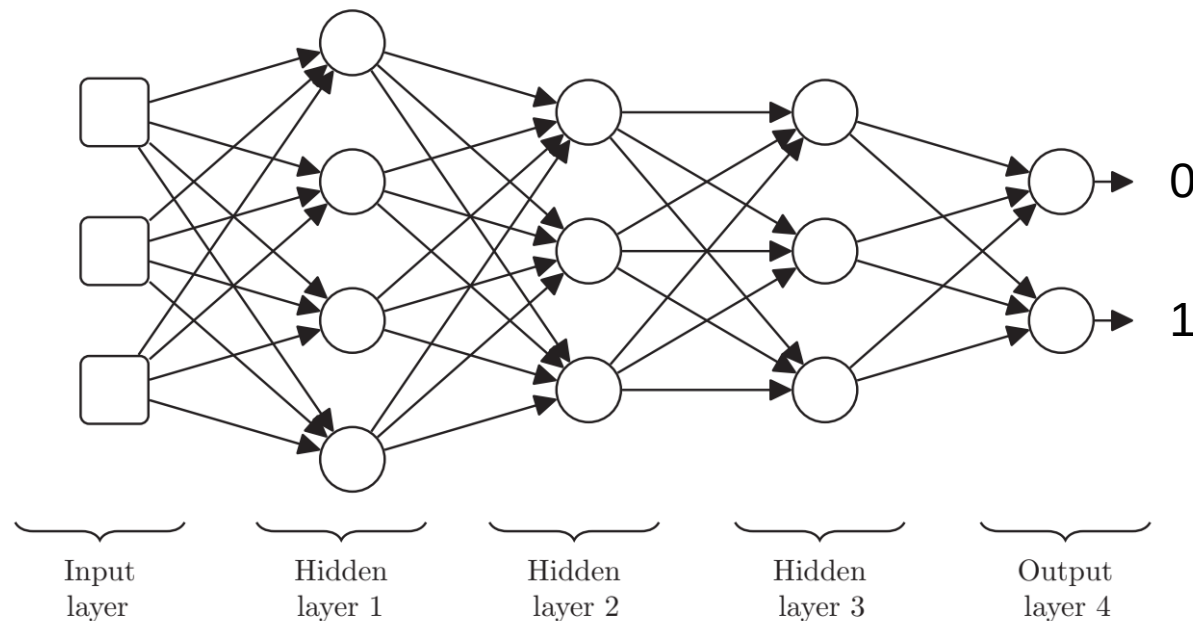
FEED FORWARD NEURAL NETWORK

- FFNN là một mạng neuron gồm 1 tầng input, nhiều tầng ẩn, 1 tầng output.
- Liên kết từ tầng thấp lên tầng cao (không có ngược lại)
- Không có liên kết giữa các neuron cùng tầng với nhau.
- Tầng ẩn cho phép học các đặc trưng quan trọng từ dữ liệu đầu vào



FEED FORWARD NEURAL NETWORK

- Dữ liệu đầu vào và đầu ra biểu diễn dạng vector.



[0, 1] : cat
[1, 0] : dog
[1, 1] : both
[0, 0] : none

Vector $x = [x_1 \ x_2 \ \dots \ x_n]$

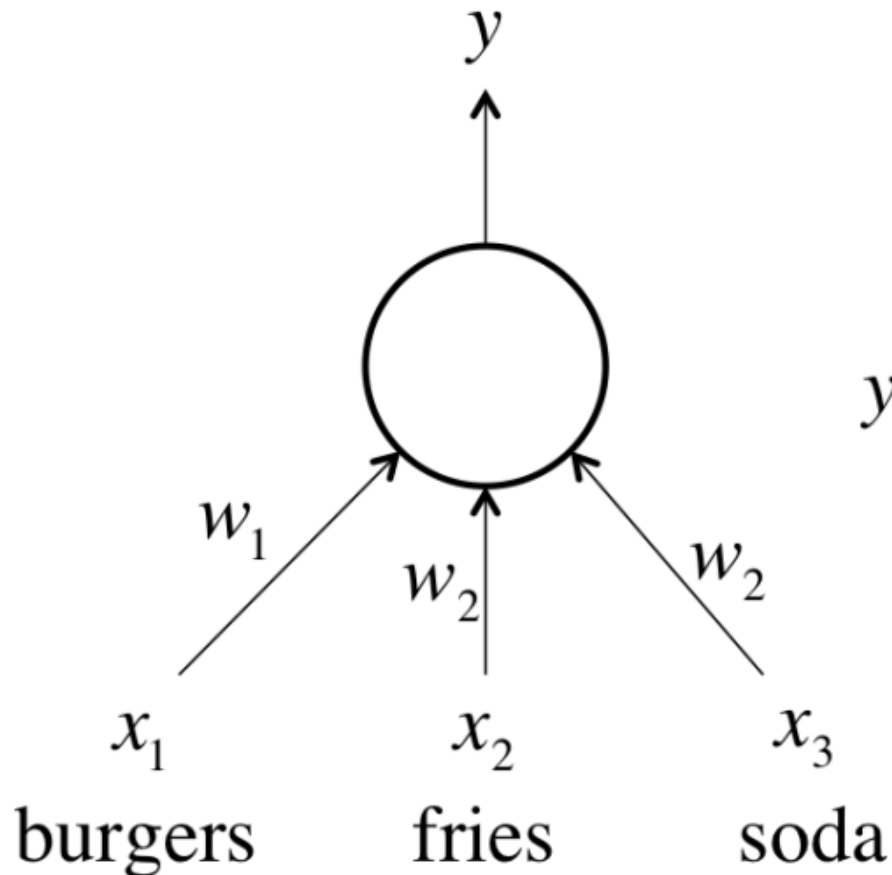
Vector $y = [y_1 \ y_2 \ \dots \ y_m]$ $\longrightarrow \mathbf{y} = f(\mathbf{W}^T \mathbf{x} + \mathbf{b})$

$W^{n \times m}$: trọng số

T là hàm chuyển vị thực hiện phép nhân ma trận

LINEAR NEURAL NETWORK

- Hầu hết các loại neuron được định nghĩa bởi hàm $f(x) = ax + b$.
- Với $a = 1$; $b = 0 \rightarrow f(x) = x$



$$y^{(i)} = w_1 x_1^{(i)} + w_2 x_2^{(i)} + w_3 x_3^{(i)}$$

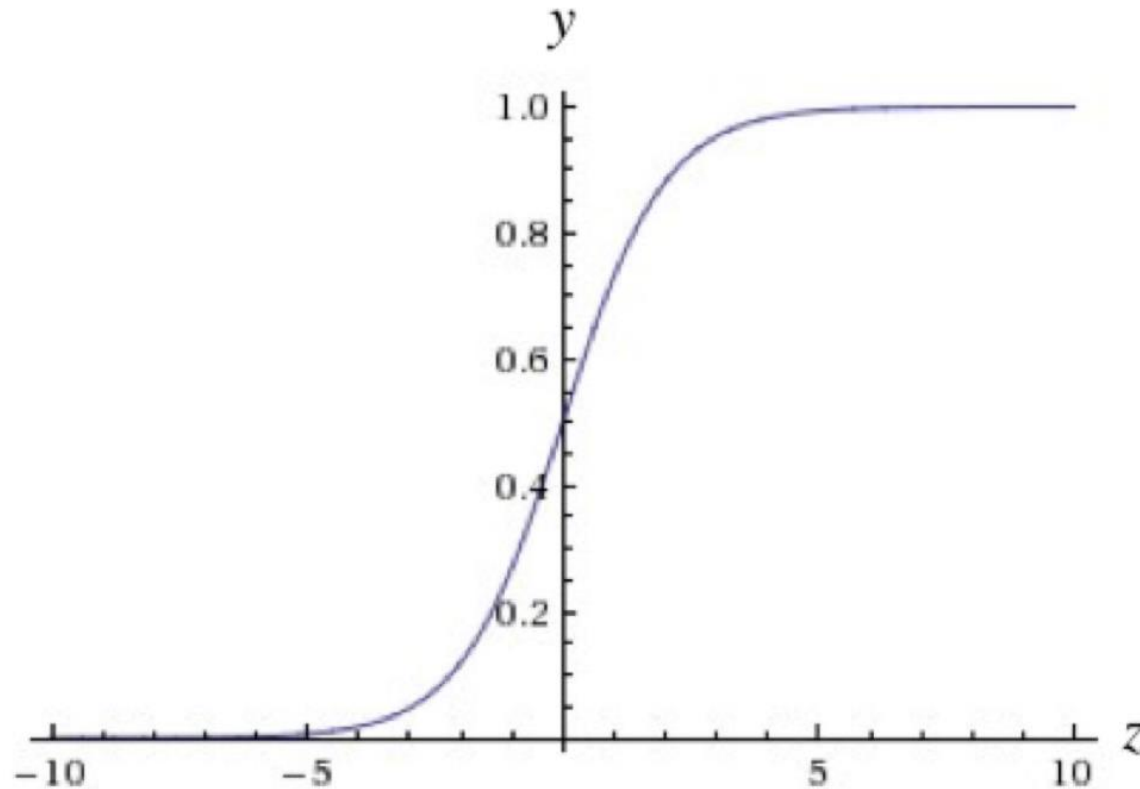
Không có tầng ẩn gọi là Linear NN
→ Không học được các mối quan hệ phức tạp
→ Cần các loại phi tuyến để học

SIGMOID - NONLINEAR NEURONS

- Hàm có dạng

$$f(z) = \frac{1}{1 + e^{-z}}$$

- Khi giá trị biến z rất nhỏ thì y gần 0, ngược lại khi z rất lớn, y tiến gần 1

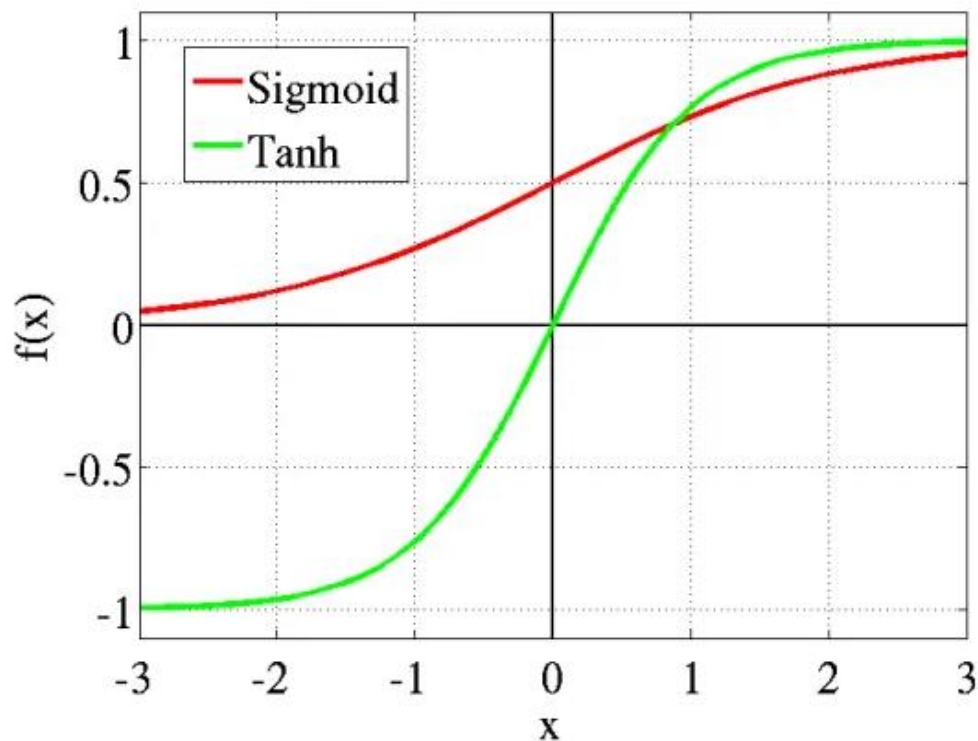


TANH - NONLINEAR NEURONS

- Hàm có dạng

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

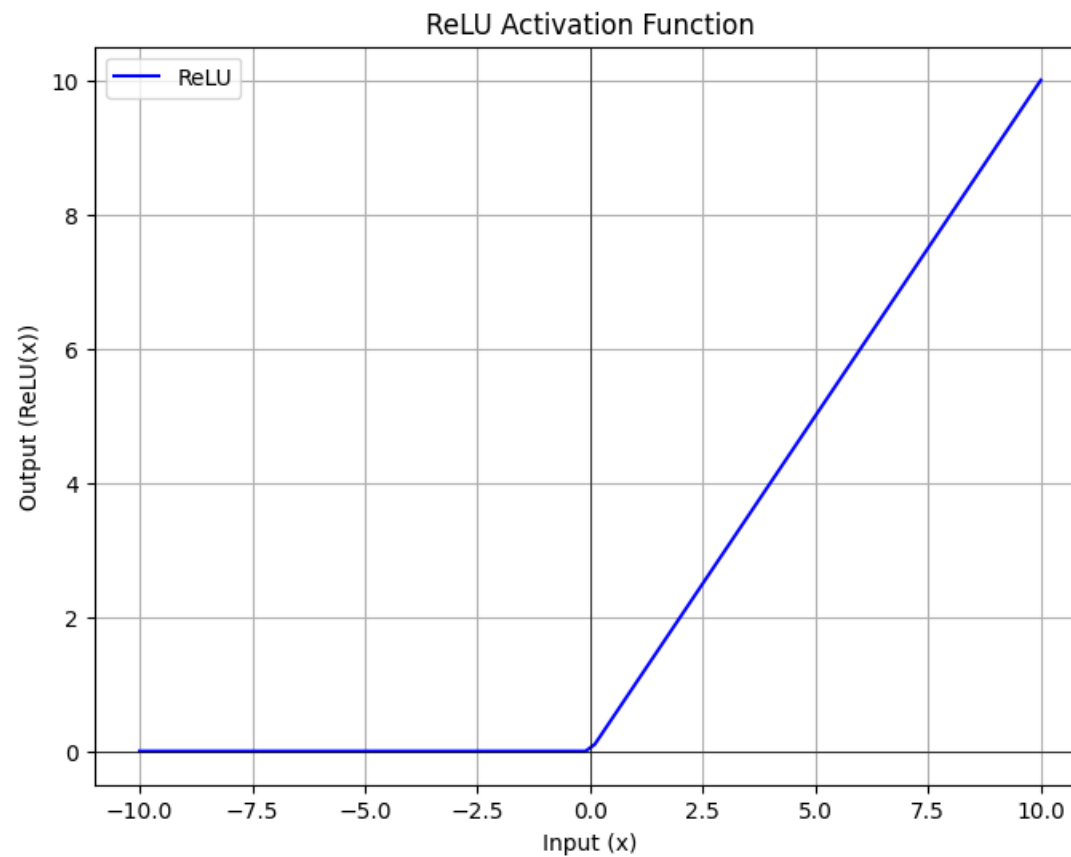
- Tương tự như Sigmoid, nhưng phạm vi $[-1; 1]$



RESTRICTED LINEAR UNIT - NONLINEAR NEURONS

- Hàm có dạng
- Phạm vi $[0; \infty)$

$$ReLU(z) = \max(0, z)$$



SOFTMAX LAYER

- Chuyển vector đầu ra thành các phân bố xác suất
- Giúp cho việc dự báo nhãn tốt hơn
- Ví dụ: nhận dạng chữ số viết tay từ 0 $\rightarrow$ 9

$$[p_0, p_1, p_2 \dots p_9] = \sum_{i=0}^9 p_i = 1$$

- Đầu ra của một neuron trên tầng softmax phụ thuộc vào đầu ra của tất cả các neuron khác trên cùng tầng với nó. Ta chuẩn hóa như sau

$$y_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- Z_i : giá trị của neuron softmax thứ i

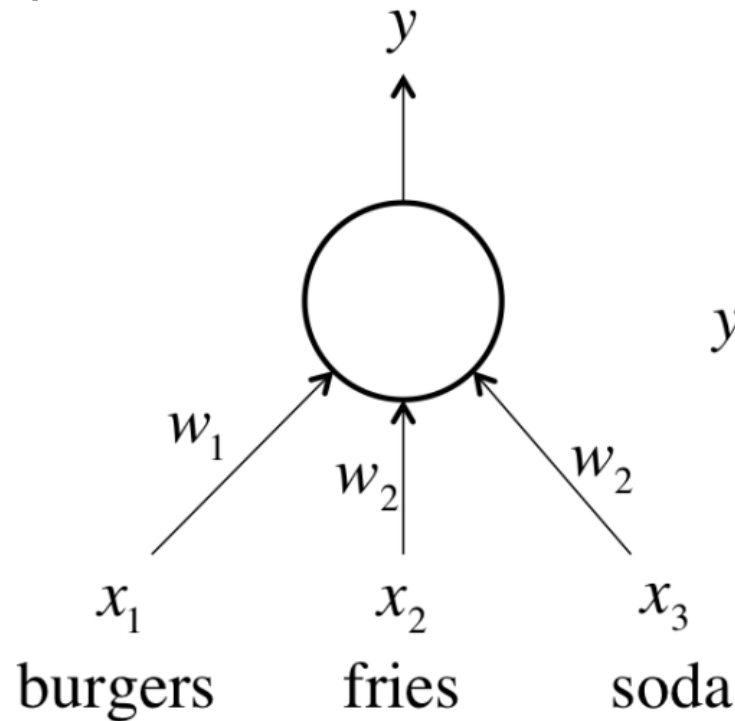
HUẤN LUYỆN FEED FORWARD NEURAL NETWORK

- Quá trình xác định vector tham số θ (weight) mô hình gọi là huấn luyện (train)
- Quá trình huấn luyện trên tập mẫu cung cấp và cập nhật liên tục tham số weight để đạt lỗi nhỏ nhất (gần 0)
- Hàm tính lỗi

$$E = \frac{1}{2} \sum_i (t^{(i)} - y^{(i)})^2$$

$t(i)$: giá trị thật sự ở mẫu i

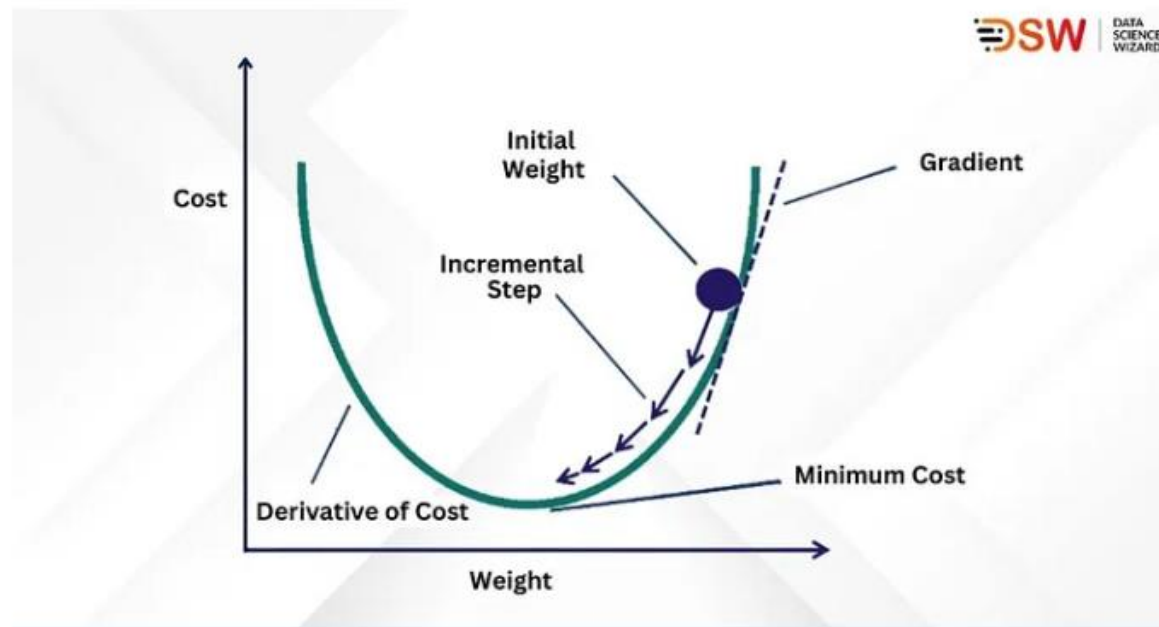
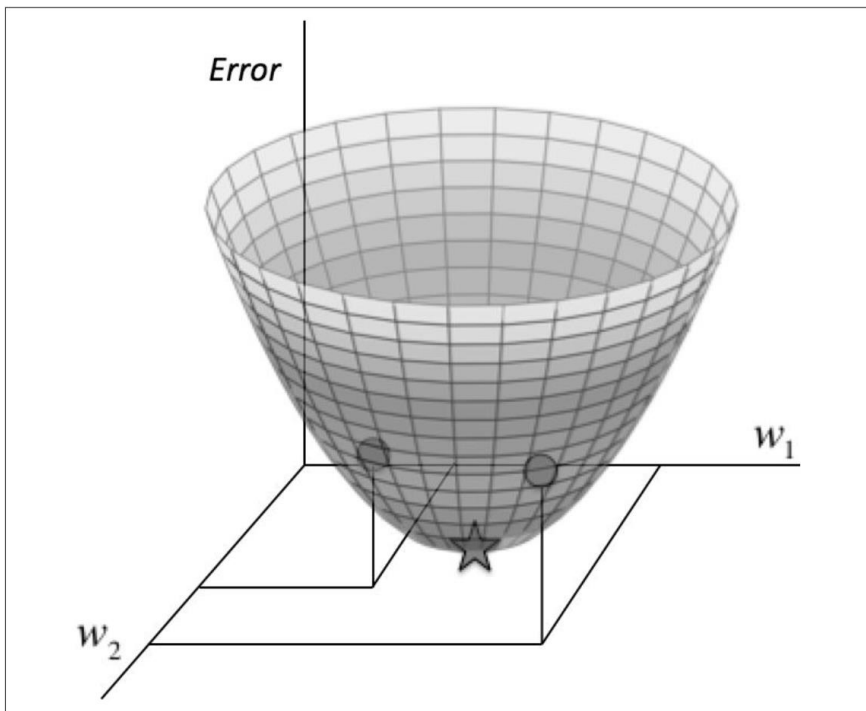
$y(i)$: giá trị do NN dự báo ở mẫu i



$$y^{(i)} = w_1 x_1^{(i)} + w_2 x_2^{(i)} + w_3 x_3^{(i)}$$

GRADIENT DESCENT

- Là giải thuật tối ưu hóa lặp tinh chỉnh tham số mô hình theo độ dốc
- Các đường đồng mức càng gần nhau thì độ dốc càng lớn
- Hướng dốc luôn vuông góc với đường viền gọi là gradient
- Xác định giá trị gradient ở mỗi vị trí để tìm hướng dốc gọi là gradient descent



GRADIENT DESCENT

- Với mỗi tham số θ được tính

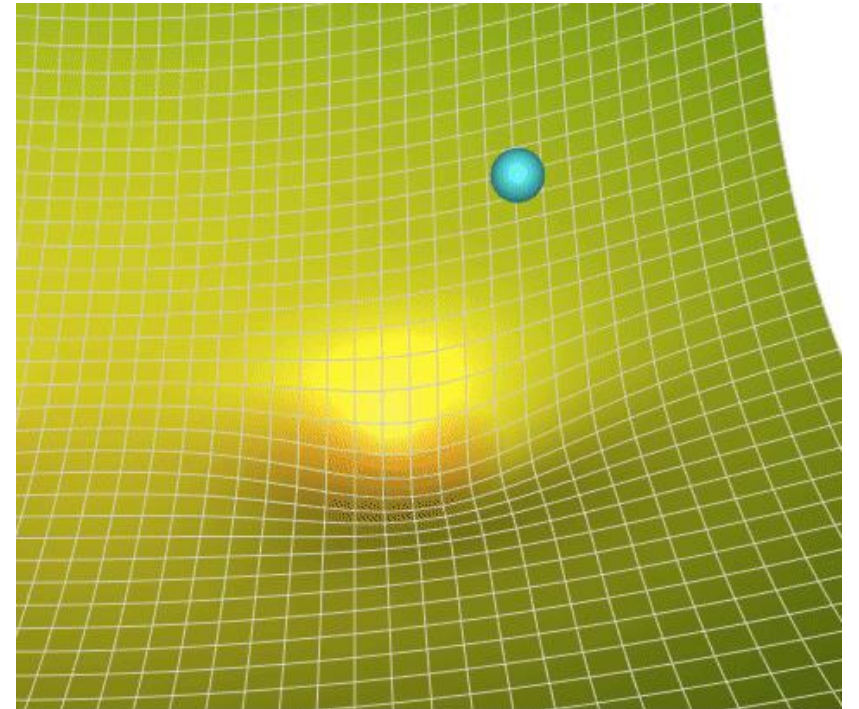
$$\delta_{n+1} = \delta_n - \text{learning rate} * f(\delta_n)$$

Learning rate (lr): hệ số tỷ lệ cho bước tiếp theo

$f(\delta_n)$: hàm gradient tính độ dốc

δ_n : gradient ở vị trí n

- *lr càng nhỏ thì GD càng lâu hội tụ và đạt được điểm tối ưu*
- *lr càng lớn thì GD không hội tụ điểm tối ưu (nhảy ra ngoài)*



GIẢI THUẬT GRADIENT DESCENT

- Ví dụ: cho hàm bậc 2 sau

$$f(x) = x^2 - 4x + 1$$

Hàm gradient $f(x)$ là

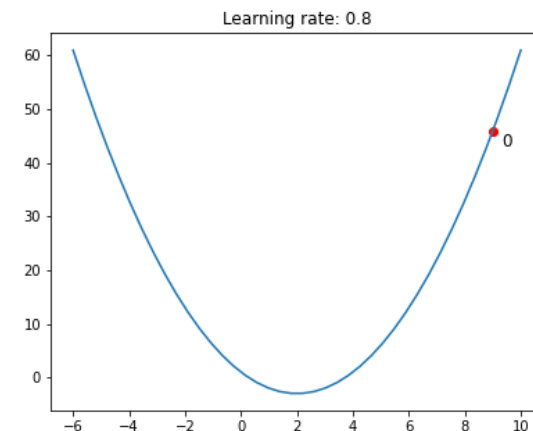
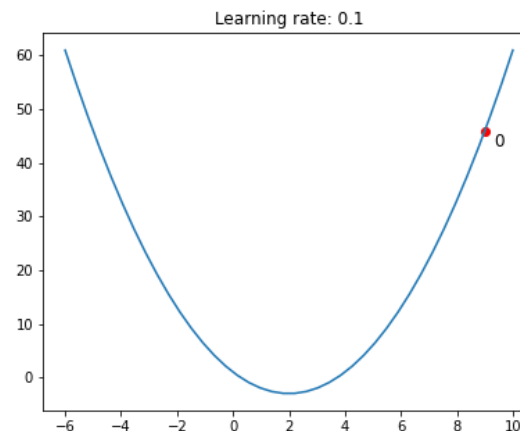
$$\frac{df(x)}{dx} = 2x - 4$$

- Epoch = 3; $x = 9$; learning rate (lr) = 0.1

$$x_1 = 9 - 0.1 * (2 * 9 - 4) = 7.6$$

$$x_2 = 7.6 - 0.1 * (2 * 7.6 - 4) = 6.48$$

$$x_3 = 6.48 - 0.1 * (2 * 6.48 - 4) = 5.584$$



BÀI TẬP GIẢI THUẬT GRADIENT DESCENT

- BT: cho hàm bậc 2 sau

$$f(x) = x^2$$

- Epoch = 10; $x = -5.6$; learning rate (lr) = 0.1

1. Hãy tính gradient descent cho 10 epochs
2. Với $x = 5$; learning rate = 0.001, tính số epochs để hội tụ
3. Với $x = -5$; learning rate = 0.0005, tính số epochs để hội tụ

GIẢI THUẬT GRADIENT DESCENT – NHIỀU BIẾN

- Cho hàm f như sau: $f(x, y) = x^2 + y^2$
- Hàm cost : $\min(f(x, y))$
- Công thức gradient: $\delta_{n+1} = \delta_n - \eta * f(x, y)$
- Giá trị tham số:

$$\delta_x = \delta_x - \eta * \frac{df(x, y)}{dx} = \delta_x - \eta * 2x$$

$$\delta_y = \delta_y - \eta * \frac{df(x, y)}{dy} = \delta_y - \eta * 2y$$

Với $\eta = 0.1$

θ_1	θ_2	$J(\theta) = \theta_1^2 + \theta_2^2$
1	1	2
0.8	0.8	1.28
0.64	0.64	0.4096
0.512	0.512	0.2621
0.409	0.409	0.1677
...	...	...
0	0	0

BÀI TẬP GIẢI THUẬT GRADIENT DESCENT – NHIỀU BIẾN

1. Cho hàm $f(x,y) = y^2 - x^2 + 2x + 2y + 4$

Với $x = 1; y = -1; lr = 0.2$; Tìm epoch hội tụ

2. Cho hàm $f(x,y) = 2x^2 - xy + 3y^2$

Với $x = 5; y = 5; lr = 0.01$; Tìm epoch hội tụ



GIẢI THUẬT BACKPROPAGATION

- Ra đời 1960s và được phổ biến bởi Rumelhart, Hinton and Williams (1989)



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

